CIE-1 STUDY GUIDE · UNIT 2 OF 3 · CS51

Unit 2
Requirements & Design

SOFTWARE ENGINEERING & MODELLING

How to find out, write down, check and manage what a system must do, and the core ideas every good software design is built on.

IN THIS UNIT

Chapter 4 · Requirements Engineering

Chapter 5 · Design Concepts

SYLLABUS

Requirements engineering: Functional and Non-functional requirements, Requirements engineering processes, Requirements elicitation, Requirements specification, Requirements validation, Requirements change.

Design Concepts: Design Within the Context of Software Engineering, Design Concepts, The Design Model.

Built from **your lecture slides**, **Sommerville (10th ed)** and **Pressman**.

Course coordinators
Mr. Pradeep Kumar D · Dr Shilpa S Chaudhari

Start here: how to use this guide

This guide covers **exactly the CIE-1 portion** of Software Engineering & Modelling (CS51): all of Unit 1, all of Unit 2, and Unit 3 up to **architectural views**. It assumes you know nothing yet. Every point from your lecture slides is here, explained in plain English first, then in the textbook wording you should use in answers.

Unit	Syllabus topics (CIE-1)	Where in this guide
Unit 1	**Introduction:** professional software development, software engineering ethics	Chapter 1 · Introduction
Unit 1	**Software processes:** process models, process activities, coping with change, process improvement	Chapter 2 · Software Processes
Unit 1	**Agile software development:** agile methods, agile development techniques, agile project management	Chapter 3 · Agile Development
Unit 2	**Requirements engineering:** functional & non-functional requirements, RE processes, elicitation, specification, validation, change	Chapter 4 · Requirements Engineering
Unit 2	**Design concepts:** design within the context of SE, design concepts, the design model	Chapter 5 · Design Concepts
Unit 3	**Architectural design:** architectural design decisions, architectural views (patterns onwards are after CIE-1)	Chapter 6 · Architectural Design

How each chapter works

1. **Opener page**: what the chapter covers, how long it takes and what you will be able to do.
2. **Sections**: each idea is explained simply, then given in exam wording, with an example and a diagram where one helps.
3. **Chapter in one page**: a dense revision sheet. Re-read it the night before the exam.
4. **Likely exam questions**: each one says what to write and which diagram to draw.

What the coloured boxes mean

DEFINITION — LEARN THIS LINE
A definition to learn **word for word**. Write it as the first line of your answer.

★ KEY POINT
A fact you must remember. These are the things examiners look for.

IN SIMPLE WORDS
The idea in everyday language. Read this first if a topic feels confusing.

EXAMPLE
A worked example, usually from the textbook case studies.

✎ EXAM TIP
How to write the answer: structure, diagram, keywords.

⚠ WATCH OUT
A common confusion or mistake that loses marks.

MEMORY TRICK
A trick for remembering a list.

? QUICK CHECK
Test yourself. Cover the answer and try it first.

Highlighted phrases are the exact keywords to use in your answers.

Priority badges

★★★ MUST KNOW

Asked often: definitions, models, diagrams, comparisons. Study these first and practise the diagrams.

★★ IMPORTANT

Commonly asked as short questions or parts of long answers. Understand them well.

★ READ ONCE

Background or lower-priority material. Read it once so nothing surprises you.

Study plans

If you have a week

- **Day 1**: Ch 1 + start Ch 2
- **Day 2**: finish Ch 2 and draw every model
- **Day 3**: Ch 3
- **Day 4**: Ch 4
- **Day 5**: Ch 5 + Ch 6
- **Day 6**: all question banks
- **Day 7**: one-page recaps + diagrams

If you have 3 days

- **Day 1**: Ch 1–3, focusing on ★★★ MUST KNOW
- **Day 2**: Ch 4–6, focusing on ★★★ MUST KNOW
- **Day 3**: question banks, then recaps; redraw every diagram once

If it's the night before

- Read all six **one-page recaps**
- Redraw the must-know diagrams from memory
- Skim each question bank and say the answer outline out loud
- Sleep; recall is better when you're rested

✎ GOLDEN RULE FOR SE ANSWERS

For anything worth more than 2 marks: **definition → diagram → explanation in points → example → advantages/disadvantages (or comparison)**. A neat, labelled diagram earns marks even when your text is short.

⚠ NOT IN CIE-1, SO SKIP THESE FOR NOW

Unit 3 from **architectural patterns** onwards (MVC, layered, repository, client–server, pipe-and-filter, application architectures), object-oriented design using UML, design patterns and implementation issues. Also all of **Unit 4** (testing, evolution) and **Unit 5** (project management, planning, quality). They come after CIE-1.

Contents · Unit 2

UNIT 2 **Requirements Engineering & Design Concepts** 4

CH 4 **Requirements Engineering** 5

4.1 Requirements, readers and stakeholders 5
4.2 Functional and non-functional requirements 9
4.3 Requirements engineering processes 16
4.4 Requirements elicitation 18
4.5 Requirements specification 25
4.6 Requirements validation 32
4.7 Requirements change 33
4.8 Lab: analysing a real-time case study 36
Chapter 4 in one page 38
Likely exam questions 39

CH 5 **Design Concepts** 41

5.1 Design within the context of software engineering 41
5.2 The design process 44
5.3 Design concepts 45
5.4 Design concepts at a glance 56
5.5 The design model 57
Chapter 5 in one page 62
Likely exam questions 63

EXAM **Exam-Day Playbook** 64

CIE-1 · UNIT 2

Unit 2: Requirements Engineering & Design Concepts

2

How to find out, write down, check and manage what a system must do, and the core ideas every good software design is built on.

SYLLABUS, WORD FOR WORD

Requirements engineering: Functional and Non-functional requirements, Requirements engineering processes, Requirements elicitation, Requirements specification, Requirements validation, Requirements change.

Design Concepts: Design Within the Context of Software Engineering, Design Concepts, The Design Model.

CHAPTERS IN THIS UNIT

CHAPTER 4 — **Requirements Engineering**
Functional & non-functional requirements · RE processes · Elicitation · Specification · Validation · Change — page 5

CHAPTER 5 — **Design Concepts**
Design in software engineering · The design process · Design concepts · The design model — page 41

COURSE OUTCOME

CO2: Analyze the functional and non-functional requirements and design concepts for the given problem.

LAB COMPONENT

Analysis of the real-time case study application.

CIE-1 SCOPE

Whole unit is in CIE-1.

UNIT 2 · CHAPTER 4

Requirements Engineering

Functional & non-functional requirements · RE processes · Elicitation · Specification · Validation · Change

STUDY TIME
About 5 hours (two sittings)

SOURCES
Your Ch 4 slides (88 slides) + Sommerville Ch 4

EXAM WEIGHT
Very high: FR vs NFR, NFR types, elicitation, SRS structure and validation are asked almost every time

AFTER THIS CHAPTER YOU CAN

- Define RE, user requirements and system requirements
- Differentiate functional, non-functional and domain requirements
- Draw the NFR classification tree with Mentcare examples
- Turn a vague goal into a verifiable NFR using metrics
- Draw the spiral RE process and the elicitation cycle
- Compare interviews, ethnography, stories and scenarios
- Write requirements as sentences, forms, tables and use cases
- Give the structure of a software requirements document
- List validation checks, techniques and review checks
- Explain requirements management and change management

4.1 Requirements, readers and stakeholders ★★★ MUST KNOW

IN SIMPLE WORDS

Before an architect draws a house, the owner says what is needed: "three bedrooms, a kitchen facing east, a total cost under 40 lakh rupees, and it must follow the city building code". The first two are *services* the house must provide; the last two are *constraints*. Software is the same. The list of services and constraints is the **requirements**; the work of finding, writing and checking that list is **requirements engineering**.

The **requirements** for a system are the descriptions of the **services** that a system should provide and the **constraints** on its operation. They reflect what customers need from a system that serves a purpose such as controlling a device, placing an order or finding information.

DEFINITION — LEARN THIS LINE

Requirements engineering (RE) — the process of establishing the services that a customer requires from a system and the constraints under which it operates and is developed.

System requirements — the descriptions of the system services and constraints that are generated during the requirements engineering process.

Sommerville's wording is also worth quoting: the process of finding out, analysing, documenting and checking these services and constraints is called requirements engineering.

What is a requirement? ★★ IMPORTANT

The term is not used consistently in industry. A requirement **may range from a high-level abstract statement of a service or of a system constraint to a detailed mathematical functional specification**. This is inevitable because requirements may serve a **dual function**:

- They may be the **basis for a bid for a contract** — therefore they must be *open to interpretation*, so that different companies can propose different solutions.
- They may be the **basis for the contract itself** — therefore they must be *defined in detail*.
- Both these statements may be called "requirements".

★ REQUIREMENTS ABSTRACTION (DAVIS)

"If a company wishes to let a contract for a large software development project, it must define its needs in a sufficiently abstract way that a solution is not pre-defined. The requirements must be written so that several contractors can bid for the contract, offering, perhaps, different ways of meeting the client organization's needs. Once a contract has been awarded, the contractor must write a system definition for the client in more detail so that the client understands and can validate what the software will do. Both of these documents may be called the requirements document for the system."

In short: an **abstract** version is written so that contractors can bid; a **detailed** version is written after the contract is awarded so that the client can check (validate) what will be built. Many problems in RE come from not separating these two levels clearly — which is why we name them separately.

User requirements and system requirements ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

User requirements — statements in natural language plus diagrams of the services the system provides and its operational constraints. Written for customers.

System requirements — a structured document setting out detailed descriptions of the system's functions, services and operational constraints. It defines what should be implemented, so it may be part of a contract between client and contractor.

Basis	User requirements	System requirements
Level of detail	High-level, abstract; may range from broad feature statements to fairly precise descriptions	Detailed, precise descriptions of functions, services and operational constraints
Notation	Natural language plus diagrams	Structured document (natural language, forms, tables, models)
Written for	Customers: client managers, end-users	Engineers: system architects, software developers
Purpose	Agree what services are needed; basis for a bid	Define exactly what is to be implemented; may be part of the contract

Basis	User requirements	System requirements
Also called	User requirements definition	System requirements specification / functional specification

Fig 4.1 shows the difference using the **Mentcare** system (a patient information system for mental health care, one of the textbook case studies). One general user requirement is expanded into five specific system requirements.

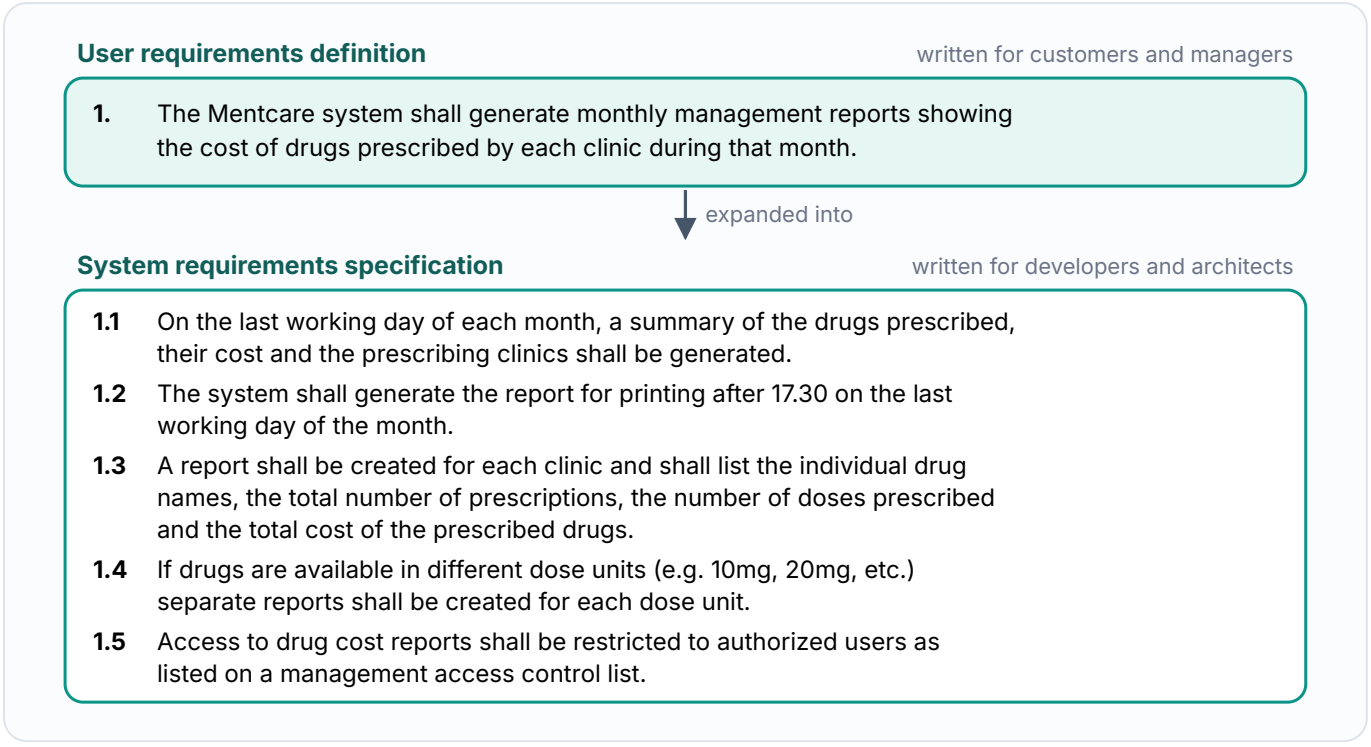


Fig 4.1 User and system requirements: one user requirement expanded into five system requirements. Sommerville Fig 4.1 · Your slides, slide 7

Requirements are written at different levels of detail because **different readers use them in different ways** (Fig 4.2). Readers of user requirements are not usually concerned with how the system will be implemented; readers of system requirements need to know more precisely what the system will do, because they must support business processes with it or are involved in implementing it.

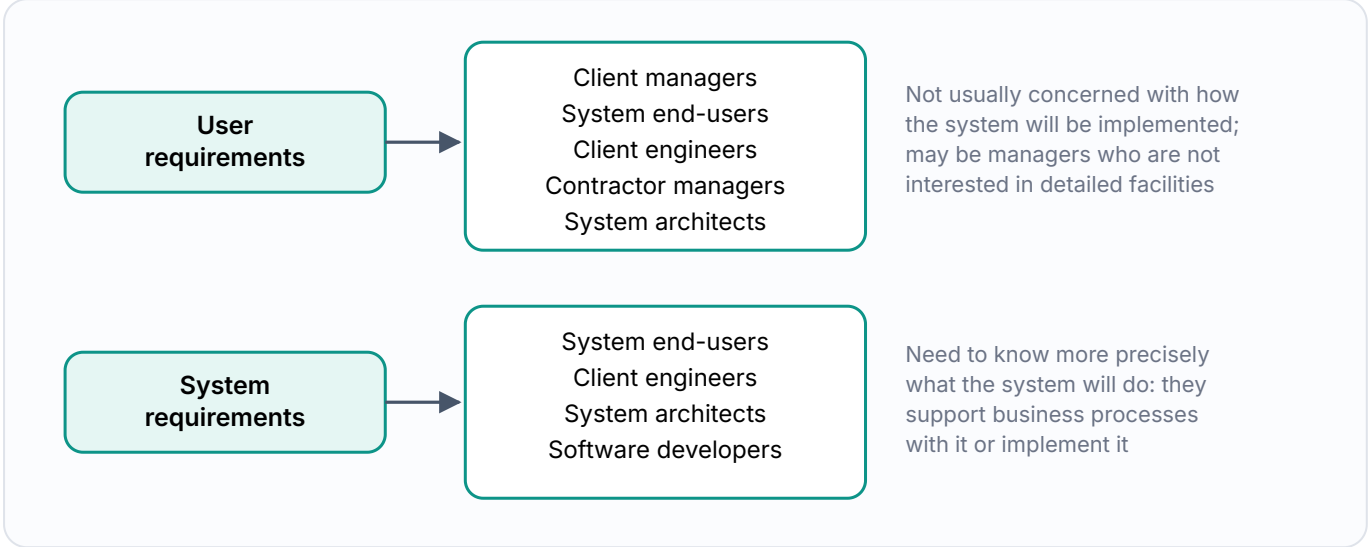


Fig 4.2 Readers of different types of requirements specification. Sommerville Fig 4.2 · Your slides, slide 8

✎ EXAM TIP

For "Differentiate user and system requirements" (4–5 marks): write both definitions, a 4-row table, the Mentcare drug-cost example (one user requirement → system requirements 1.1–1.5) and draw Fig 4.2. The figure alone earns easy marks.

System stakeholders ★★★ MUST KNOW

The readers in Fig 4.2 are examples of **stakeholders**. Besides users, many other people have an interest in a system.

DEFINITION — LEARN THIS LINE

System stakeholder — any person or organization who is affected by the system in some way and so who has a legitimate interest in it.

Stakeholder types (range from end-users through managers to external stakeholders):

- **End users** — the people who use the system in their daily work.
- **System managers** — the people who manage and run the system.
- **System owners** — the organisation that pays for and owns the system.
- **External stakeholders** — e.g. regulators, who certify the acceptability of the system.

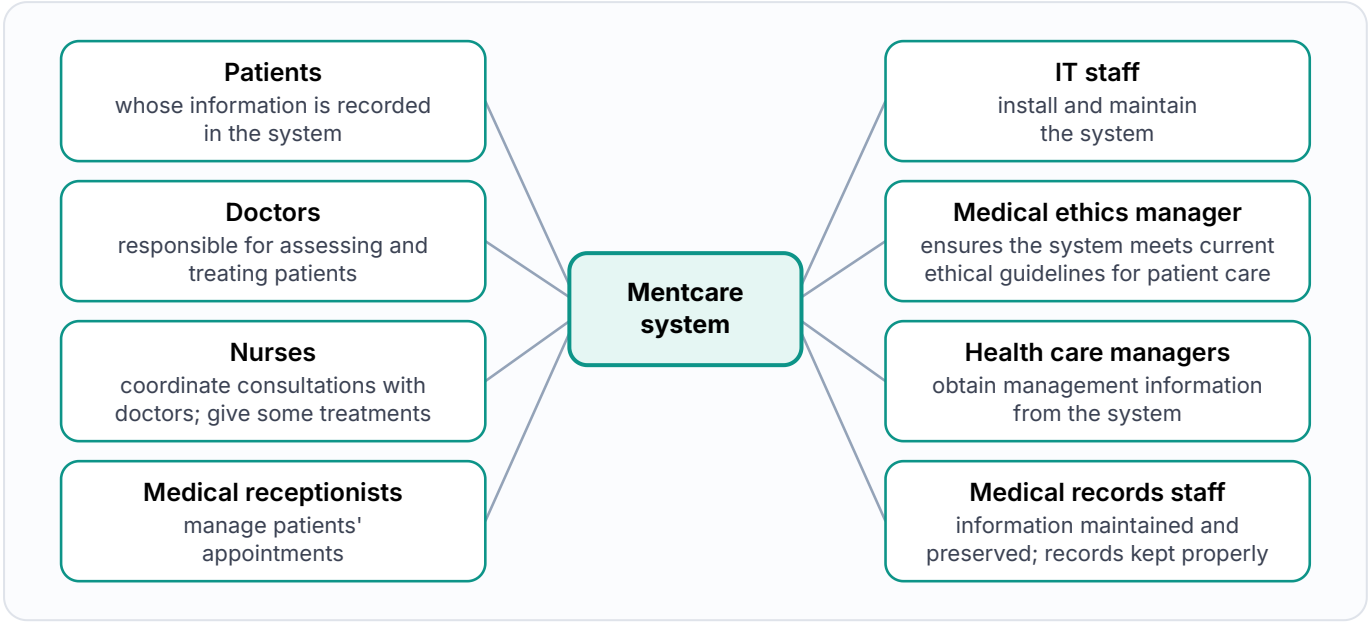


Fig 4.3 The eight stakeholder groups of the Mentcare system (the textbook also includes patients' relatives). Drawn for this guide from your slides, slides 10–11

Before RE starts: the feasibility study ★ READ ONCE

Some understanding of the requirements is needed before deciding to build or buy a system at all. This early RE gives a high-level view of what the system might do and its benefits, which is assessed in a **feasibility study** — a short, focused study early in the RE process that answers three questions: (1) Does the system contribute to the overall objectives of the organisation? (2) Can it be implemented within schedule and budget using current technology? (3) Can it be integrated with other systems that are used? If any answer is "no", you should probably not go ahead.

Agile methods and requirements ★★ IMPORTANT

- Many agile methods argue that producing detailed system requirements is a **waste of time**, as requirements change so quickly.
- The requirements document is therefore **always out of date**.
- Agile methods usually use **incremental requirements engineering** and may express requirements as **user stories** (Chapter 3), written on cards or whiteboards; the user prioritises the stories for the next increment.
- This is practical for **business systems** but problematic for systems that require **pre-delivery analysis** (e.g. critical systems) or systems developed by **several teams**.

Sommerville's middle path: even with agile development, write a **short supporting document** that defines the business and dependability requirements, because requirements that apply to the whole system are easily forgotten when you focus on the features of the next release.

4.2 Functional and non-functional requirements ★★★ MUST KNOW

IN SIMPLE WORDS

Functional = what the system *does* (search, generate a report, identify a user). **Non-functional** = how well it must do it and under which rules (how fast, how reliable, how secure, which laws and standards apply). For a car, "the car shall have brakes" is functional; "it shall stop from 100 km/h within 40 metres" and "it shall meet emission laws" are non-functional.

DEFINITION — LEARN THIS LINE

Functional requirements — statements of services the system should provide, how the system should react to particular inputs and how the system should behave in particular situations. They may also state what the system should *not* do.

Non-functional requirements — constraints on the services or functions offered by the system, such as timing constraints, constraints on the development process, standards, etc. They often apply to the system as a whole rather than to individual features or services.

Domain requirements — constraints on the system from the domain of operation (the application area, e.g. medicine, banking, railways).

The line between them is not sharp. A user requirement about security, such as limiting access to authorised users, looks non-functional. When developed in more detail it generates requirements that are clearly functional, such as including user authentication facilities. Requirements are not independent: one requirement often **generates or constrains** other requirements (Fig 4.4).

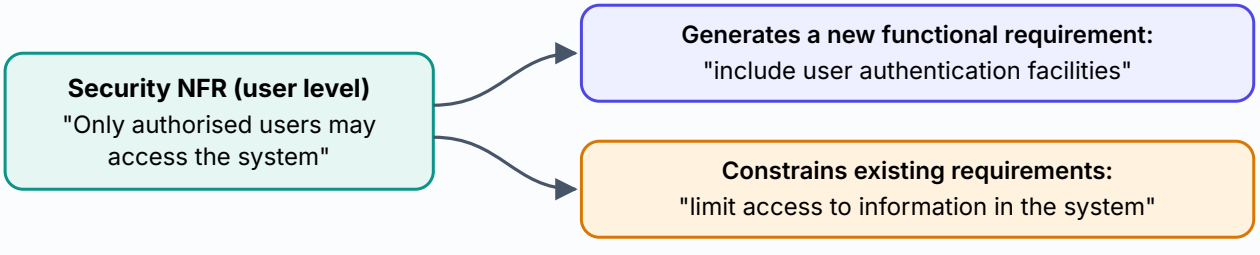


Fig 4.4 One non-functional requirement can generate new functional requirements and restrict existing ones. Drawn for this guide from Sommerville 4.1

Functional requirements ★★★ MUST KNOW

- Describe **functionality or system services** — what the system should do. They are statements of the services the system must provide, or descriptions of **how some computations must be carried out**.
- Depend on the **type of software**, the **expected users** and the **type of system** where the software is used.
- **Functional user requirements** may be high-level statements of what the system should do. They are written in natural language so that users and managers can understand them.
- **Functional system requirements** should describe the system services in detail — the functions, their inputs and outputs, and exceptions. They are written for developers.

> **EXAMPLE — MENTCARE FUNCTIONAL REQUIREMENTS**
>
> 1. A user shall be able to search the appointments lists for all clinics.
> 2. The system shall generate each day, for each clinic, a list of patients who are expected to attend appointments that day.
> 3. Each staff member using the system shall be uniquely identified by his or her 8-digit employee number.
>
> Functional requirements can be written at different levels of detail: requirement 1 is general, requirement 3 is very specific.

Information requirements: if an off-the-shelf product will meet the need, a detailed functional specification is pointless. Focus instead on **information requirements** — what information people need to do their work and how it is delivered and organised (e.g. what goes into the daily list of expected patients).

Domain requirements ★★ IMPORTANT

Domain requirements are derived from the **application domain** of the system rather than from the specific needs of system users. They may:

- be **new functional requirements** in their own right,
- **constrain existing** functional requirements, or
- set out **how particular computations must be carried out**.

The problem with domain requirements: software engineers may not understand the characteristics of the domain, so they may not know whether a domain requirement has been **missed out** or **conflicts** with other requirements. Domain requirements are found during requirements discovery, from stakeholders and from documentation (Section 4.4). For instance, the insulin pump's rule for computing a dose from the rate of change of blood sugar (Section 4.5) comes from medical knowledge, not from a user's personal wish — it sets out how a computation must be carried out.

Requirements imprecision ★★★ MUST KNOW

- Problems arise when functional requirements are **not precisely stated**.
- **Ambiguous** requirements may be interpreted in different ways by developers and users.
- A developer naturally chooses the interpretation that is **simplest to implement**. If that is not what the customer wanted, new requirements and changes are needed — causing disputes, **delayed delivery and higher cost**.

Consider the term **"search"** in Mentcare requirement 1:

User intention

Search for a patient name across **all appointments in all clinics**. Rationale: patients with mental health problems are sometimes confused and may go to a different clinic from the one booked.

Developer interpretation

Search for a patient name in **an individual clinic**: the user chooses a clinic, then searches. Easier to build, but needs more user input and takes longer.

Requirements completeness and consistency ★★★ MUST KNOW

In principle, requirements should be both complete and consistent:

Complete They should include descriptions of all facilities (services and information) required.

Consistent There should be no conflicts or contradictions in the descriptions of the system facilities.

In practice, because of system and environmental complexity, it is **impossible to produce a complete and consistent requirements document** (except for very small systems). Two reasons: (1) it is easy to make **mistakes and omissions** when writing specifications for large, complex systems; (2) large systems have **many stakeholders** with different backgrounds and often inconsistent needs, and the inconsistencies may only appear after deeper analysis or during development.

Non-functional requirements ★★★ MUST KNOW

- NFRs define **system properties and constraints**, e.g. reliability, response time and storage requirements. Constraints are I/O device capability, system representations (the data representations used in interfaces), etc.
- **Process requirements** may also be specified, mandating a particular IDE, programming language or development method.
- NFRs often relate to **emergent properties** — properties of the system as a whole (such as reliability, response time, memory use) rather than of one component.
- NFRs may be more critical than functional requirements. If these are not met, the system may be useless.

> **EXAMPLE**
>
> Users can usually find a way round a function that does not quite meet their needs, but not round a failed NFR. If an **aircraft** system does not meet its reliability requirements, it will not be certified as safe for operation; if an **embedded control system** fails to meet its performance requirements, the control functions will not operate correctly.

Non-functional requirements implementation

You can usually point to the component that implements a functional requirement (e.g. a formatting component for reports). NFRs are harder to locate — their implementation may be spread throughout the system, for two reasons:

1. NFRs may affect the **overall architecture** of a system rather than the individual components. For example, to ensure that performance requirements are met, you may have to organise the system to minimise communications between components.
2. A single NFR, such as a **security requirement**, may generate a number of related **functional requirements** that define system services that are required. It may also generate requirements that **restrict existing requirements** (Fig 4.4).

Types of non-functional requirement ★★★ MUST KNOW

NFRs arise from user needs because of budget constraints, organisational policies, the need for interoperability with other software or hardware systems, or external factors such as safety regulations or privacy legislation. Sommerville classifies them by **where they come from**: the product, the organisation, or outside (Fig 4.5).

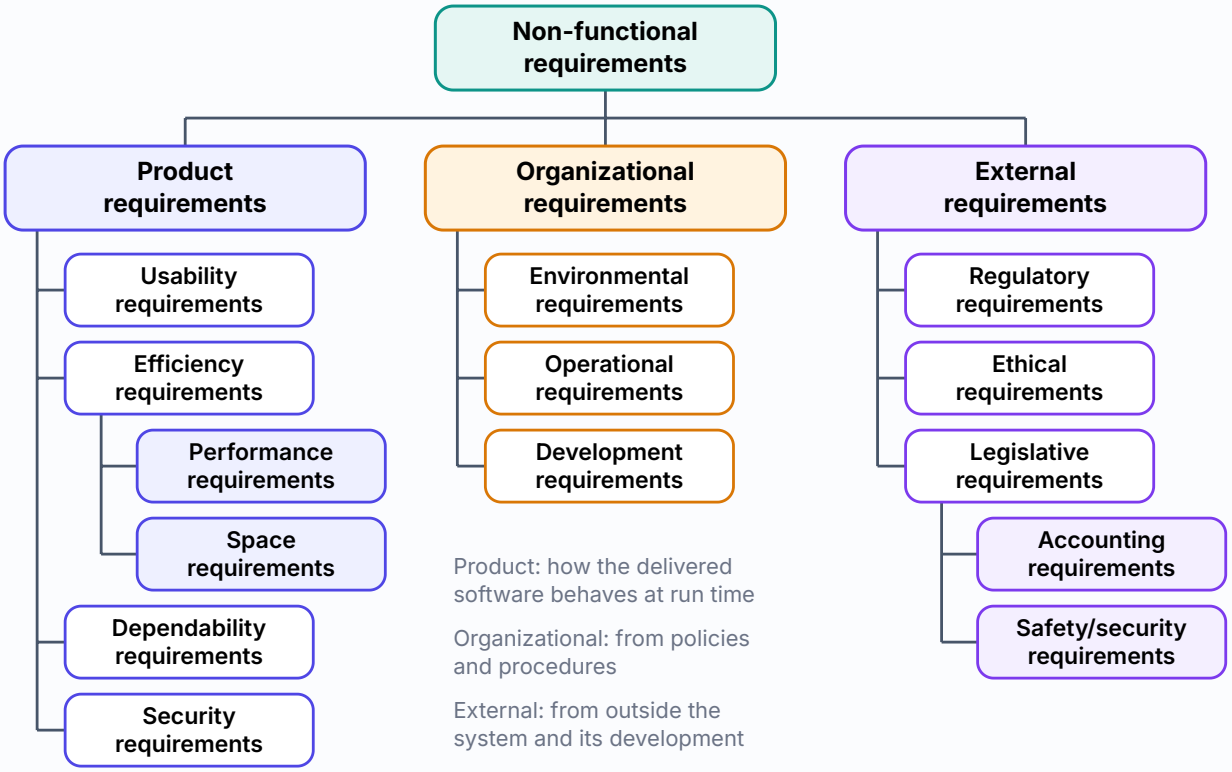


Fig 4.5 Types of non-functional requirement. Shaded boxes are sub-types (Efficiency → Performance, Space; Legislative → Accounting, Safety/security). Sommerville Fig 4.3 · Your slides, slide 20

Class	Meaning (exam wording)	Sub-types and examples
Product requirements	Requirements which specify that the delivered product must behave in a particular way — they specify or constrain the run-time behaviour of the software.	**Usability**; **efficiency** (**performance** — how fast it executes; **space** — how much memory it needs); **dependability** (e.g. reliability — acceptable failure rate); **security**. E.g. execution speed, reliability.
Organisational requirements	Requirements which are a consequence of organisational policies and procedures (in the customer's and developer's organisations).	**Operational** — how the system will be used; **development** — programming language, development environment, process standards; **environmental** — the operating environment of the system. E.g. process standards used, implementation requirements.
External requirements	Requirements which arise from factors which are external to the system and its development process.	**Regulatory** — what must be done for a regulator (e.g. a nuclear safety authority) to approve it; **legislative** — operate within the law (accounting, safety/security); **ethical** — acceptable to users and the general public. E.g. interoperability requirements, legislative requirements.

MEMORY TRICK

Three roots — **P-O-E**: what the **P**roduct does at run time, what the **O**rganisation's rules say, what the world **E**xternal to the project demands.
Leaves: Product → **U-E-D-S** "*U Eat Dosa Slowly*" (Usability, Efficiency [Performance, Space], Dependability, Security) · Organisational → **E-O-D** "*Every Office Decides*" (Environmental, Operational, Development) · External → **R-E-L** "the *REaL* world" (Regulatory, Ethical, Legislative [Accounting, Safety/security]).

Examples of non-functional requirements in the Mentcare system

Type	Requirement	Why it is this type
Product requirement	The Mentcare system shall be available to all clinics during normal working hours (Mon–Fri, 08:30–17:30). Downtime within normal working hours shall not exceed five seconds in any one day.	An **availability** requirement: when the system must be available and the allowed downtime. It says nothing about functionality — it is a constraint for the designers.
Organisational requirement	Users of the Mentcare system shall authenticate themselves using their health authority identity card.	Comes from **policy**: the health authority is moving to a standard procedure where users swipe an identity card instead of using a login name.
External requirement	The system shall implement patient privacy provisions as set out in HStan-03-2006-priv.	Comes from **privacy legislation**: the system must follow a national privacy standard.

Goals and verifiable requirements ★★★ MUST KNOW

Non-functional requirements may be very difficult to state precisely, and imprecise requirements may be difficult to verify. Stakeholders often propose NFRs as general **goals** — ease of use, the ability to recover from failure, rapid user response. Goals set out good intentions but leave room for interpretation and for dispute once the system is delivered.

DEFINITION — LEARN THIS LINE

Goal — a general intention of the user, such as ease of use.

Verifiable non-functional requirement — a statement using some measure that can be objectively tested.

Goals are still helpful to developers, as they convey the intentions of the system users. Whenever possible, though, write NFRs **quantitatively** so they can be objectively tested.

Usability requirement	Statement
Goal (not testable)	"The system should be easy to use by medical staff and should be organized in such a way that user errors are minimized."
Testable NFR	"Medical staff shall be able to use all the system functions after four hours of training. After this training, the average number of errors made by experienced users shall not exceed two per hour of system use."

⚠ WATCH OUT

Your slides say **four** hours of training; the 10th-edition textbook text says **two** hours. Either is accepted — what matters is that the rewritten requirement has **measurable numbers** (training time, errors per hour). These can be checked by software instrumentation that counts user errors during testing.

Why quantifying NFRs is hard: customers find it difficult to translate goals into measurable requirements; for some goals, such as **maintainability**, there are no simple metrics; customers may not relate a number (e.g. for reliability) to their everyday experience; and objectively verifying measurable NFRs can be very expensive.

Metrics for specifying non-functional requirements ★★★ MUST KNOW

These metrics can be measured while the system is being tested, to check whether it meets its NFRs.

Property	Measure	What it tells you
Speed	Processed transactions/second · User/event response time · Screen refresh time	How quickly the system works
Size	Mbytes · Number of ROM chips	How much memory or storage it needs
Ease of use	Training time · Number of help frames	How fast people learn it; how much help they need
Reliability	Mean time to failure · Probability of unavailability · Rate of failure occurrence · Availability	How rarely it fails; how often it is available for use
Robustness	Time to restart after failure · Percentage of events causing failure · Probability of data corruption on failure	How well it copes with and recovers from failure
Portability	Percentage of target dependent statements · Number of target systems	How easily it moves to other platforms

MEMORY TRICK

"**S**ix **S**mart **E**ngineers **R**arely **R**un **P**oorly" → Speed, Size, Ease of use, Reliability, Robustness, Portability.

NFR conflicts and where to put NFRs

- NFRs often **conflict and interact** with other requirements. The Mentcare ID-card requirement needs a card reader on each computer, but another requirement may ask for mobile access from doctors' and nurses' tablets or smartphones, which have no card readers — so an alternative identification method may be needed.
- It is difficult to separate FRs and NFRs in the requirements document. If NFRs are stated separately, relationships are hard to see; ideally, **highlight** NFRs related to emergent properties (performance, reliability) in a separate section or by marking them.
- NFRs such as reliability, safety and confidentiality are particularly important for **critical systems**.

✎ EXAM TIP

A verifiable NFR = **property + measure + number + unit + condition**. For example: "The system shall display the results of a patient-name search within 2 seconds for 95% of searches when 50 users are logged in." Examiners reward the number.

Functional vs non-functional requirements — the comparison ★★★ MUST KNOW

Basis	Functional requirements	Non-functional requirements
Meaning	Services the system should provide; how it reacts to inputs and behaves in particular situations	Constraints on the services or functions; properties of the system
Question answered	What should the system do?	How well, and under what constraints, must it do it?
Scope	Usually one feature or service	Often the system as a whole (emergent properties)
Classification	Functional user requirements and functional system requirements	Product, organisational, external
Mentcare example	Search appointment lists for all clinics; daily list of expected patients; staff identified by 8-digit number	Available 08:30–17:30 with ≤ 5 s downtime per day; ID-card authentication; privacy standard HStan-03-2006-priv
Where implemented	Can usually be traced to particular components	Spread through the system; may shape the whole architecture
If not met	Users can often work around the gap	System may be useless (e.g. not certified as safe)
How verified	Test that the function gives the right output for given inputs	Measure with metrics: response time, MTTF, training time, etc.
Ease of stating	Easier, but must avoid ambiguity ("search")	Harder; often stated as vague goals that must be made measurable

FR vs NFR for an app you use every day ★★ IMPORTANT

Practise on a **food-delivery app**. These requirements are illustrations written for this guide, not textbook text.

#	Requirement	Kind	NFR type (Fig 4.5)
1	The app shall let a customer search restaurants by restaurant name and by dish.	FR	—
2	The app shall show the live location of the delivery partner after the order is picked up.	FR	—
3	The app shall let a customer cancel an order before the restaurant accepts it.	FR	—
4	Search results shall appear within 2 seconds for 95% of searches.	NFR	Product → efficiency (performance)
5	The ordering service shall be available 99.5% of the time in each month.	NFR	Product → dependability

#	Requirement	Kind	NFR type (Fig 4.5)
6	A first-time user shall be able to place an order within 3 minutes without help.	NFR	Product → usability
7	The app shall be developed in Kotlin following the company coding standard.	NFR	Organisational → development
8	Card payments shall comply with the national payment-data regulations.	NFR	External → regulatory / legislative

? QUICK CHECK

"The system shall print a monthly drug-cost report" — FR or NFR? And "Each report shall be generated within 10 seconds"?

Answer: The first is functional (a service). The second is a non-functional product requirement (efficiency → performance); it is verifiable because it contains a number.

4.3 Requirements engineering processes ★★★ MUST KNOW

The processes used for RE **vary widely** depending on the application domain, the people involved and the organisation developing the requirements. However, there are a number of **generic activities common to all processes**:

1. **Requirements elicitation** — discovering requirements by working with stakeholders.
2. **Requirements analysis** — classifying, organising, prioritising and negotiating them (done together with elicitation).
3. **Requirements validation** — checking that the requirements define the system the customer really wants.
4. **Requirements management** — managing changes to the requirements.

The textbook describes the same work as **three key activities**: discovering requirements by interacting with stakeholders (**elicitation and analysis**); converting them into a standard form (**specification**); and checking that they define the system the customer wants (**validation**) — with **requirements management** handling change. In practice, RE is an iterative activity in which these processes are interleaved.

⚠ WATCH OUT

Slides: 4 generic activities (elicitation, analysis, validation, management). Textbook: 3 key activities (elicitation & analysis, specification, validation). In the exam write **elicitation & analysis, specification, validation and management** — that covers both versions.

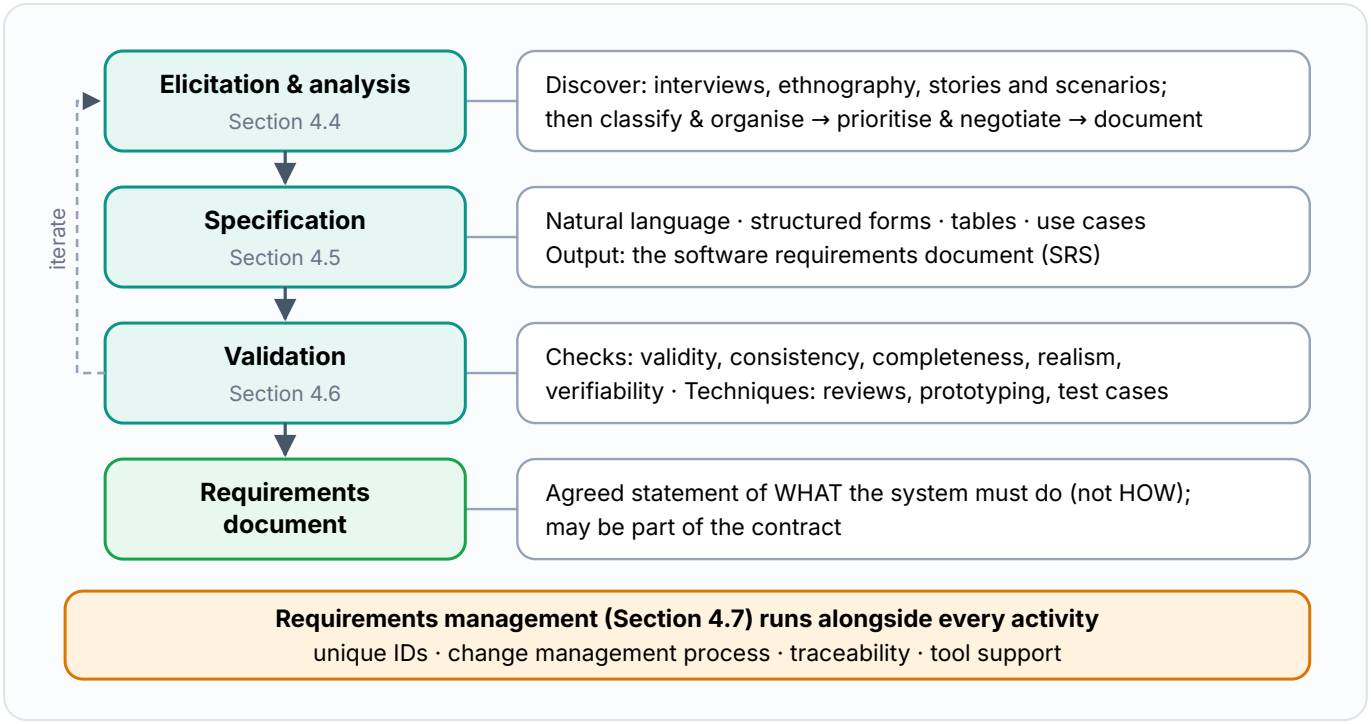


Fig 4.6 Requirements engineering at a glance — a map of this chapter. Drawn for this guide

A spiral view of the requirements engineering process ★★★ MUST KNOW

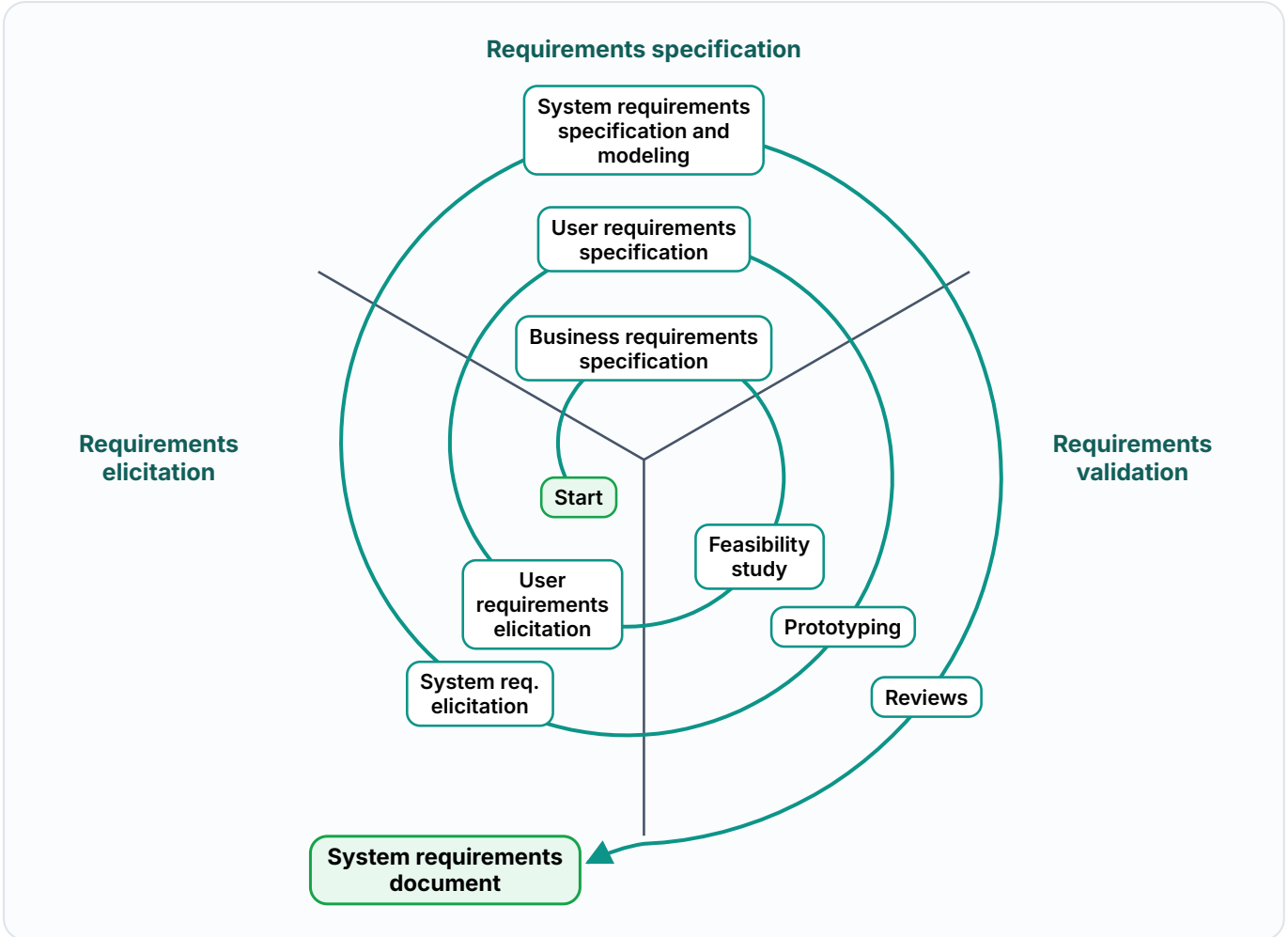


Fig 4.7 A spiral view of the requirements engineering process: each loop runs elicitation → specification → validation at a greater level of detail. Sommerville Fig 4.6 · Your slides, slide 29

- The three activities — **elicitation, specification and validation** — are organised as an iterative process around a **spiral**. Each loop adds detail: **business → user → system** requirements.
- The **output** of the RE process is a **system requirements document**.
- The time and effort spent on each activity in an iteration depends on the **stage** of the overall process, the **type of system** being developed and the **budget** available.
- **Early** in the process, most effort goes on understanding high-level **business and non-functional requirements** and the **user requirements**. **Later**, in the outer rings, more effort goes on eliciting and understanding the NFRs and more **detailed system requirements**.
- The number of iterations can vary: the spiral can be **exited** after some or all of the user requirements have been elicited. **Agile development** can be used instead of prototyping, so that the requirements and the implementation are developed together.
- In virtually all systems, **requirements change** (people understand their needs better, the buying organisation changes, the hardware, software and organisational environment changes). Changes must be managed — see Section 4.7.

> ✎ **EXAM TIP**
> To draw Fig 4.7 quickly: draw a "Y" (three lines from a centre); label the sectors **specification** (top), **elicitation** (left), **validation** (right); draw a spiral outwards from the centre; write the nine activities along it in order — *Start → business requirements spec → feasibility study → user requirements elicitation → user requirements spec → prototyping → system requirements elicitation → system requirements spec & modelling → reviews* — and end with an arrow to "System requirements document".

4.4 Requirements elicitation ★★★ MUST KNOW

> **IN SIMPLE WORDS**
> "Elicit" means to draw something out. Requirements rarely arrive written down — you draw them out of people by talking to them, watching them work and discussing realistic examples with them.

Requirements elicitation and analysis (sometimes called requirements elicitation or requirements discovery):

- Involves technical staff working with customers to find out about the **application domain**, the **services** that the system should provide and the system's **operational constraints**.
- May involve end-users, managers, engineers involved in maintenance, domain experts, trade unions, etc. These are called **stakeholders**.

The aim is to understand the **work that stakeholders do** and how they might use a new system to support it. Software engineers work with a range of system stakeholders to find out about the application domain, work activities, the services and features wanted, the **required system performance**, **hardware constraints**, **other systems**, etc. The stages are: requirements **discovery**; requirements **classification and organisation**; requirements **prioritisation and negotiation**; and requirements **specification**.

Problems of requirements elicitation ★★★ MUST KNOW

Problem (slide wording)	What it means (textbook detail)
1. Stakeholders don't know what they really want	They know only in the most general terms, find it hard to say what the system should do, and may make unrealistic demands because they don't know what is feasible.
2. Stakeholders express requirements in their own terms	They use their own language and implicit knowledge of their work; requirements engineers without domain experience may not understand them.
3. Different stakeholders may have conflicting requirements	Diverse stakeholders express requirements in different ways; engineers must find all sources and discover commonalities and conflicts.
4. Organisational and political factors may influence the requirements	E.g. managers may demand specific requirements because these will increase their influence in the organisation.
5. Requirements change during the analysis process	The economic and business environment is dynamic; the importance of requirements changes; new stakeholders may emerge who were not originally consulted.

The requirements elicitation and analysis process ★★★ MUST KNOW

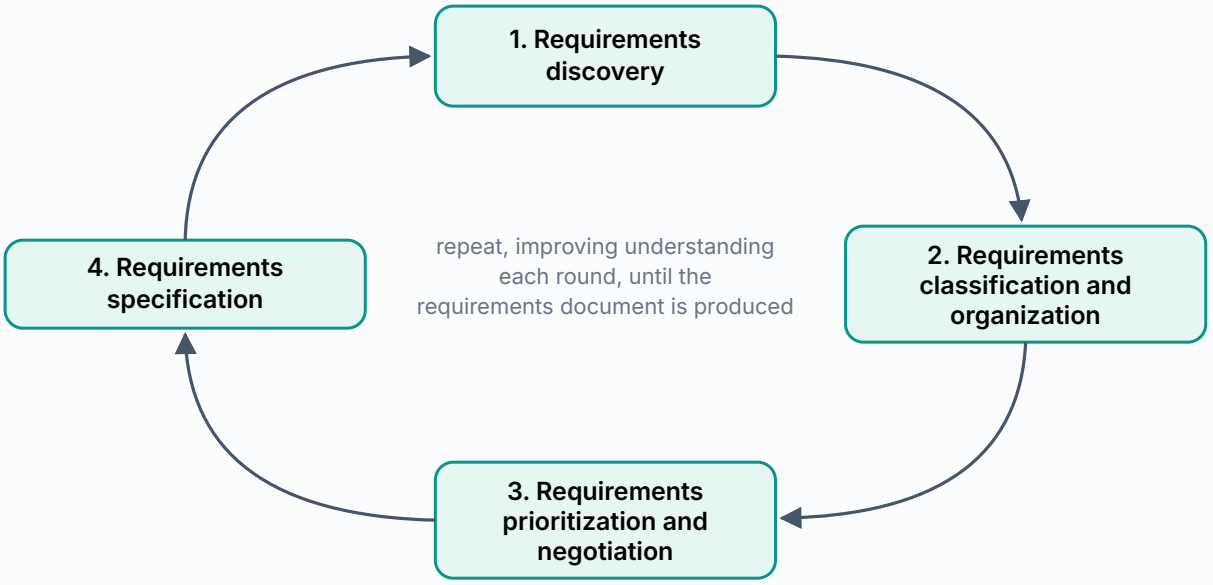


Fig 4.8 The requirements elicitation and analysis process (step 4 is called "requirements documentation" in the 10th-edition textbook). Sommerville Fig 4.7 · Your slides, slide 35

1. **Requirements discovery** — interacting with stakeholders to discover their requirements. Domain requirements (from stakeholders and documentation) are also discovered at this stage.
2. **Requirements classification and organisation** — takes the unstructured collection of requirements, groups related requirements and organises them into coherent clusters.
3. **Prioritisation and negotiation** — prioritising requirements and resolving requirements conflicts. With many stakeholders, requirements will conflict; stakeholders usually meet to agree compromise requirements.

4. **Requirements specification** — requirements are documented and input into the next round of the spiral. An early draft of the requirements document may be produced, or requirements kept informally on whiteboards, wikis or other shared spaces.

- The process is **iterative**, with continual feedback between activities. It starts with discovery and ends with documentation; the analyst's understanding improves each round; it ends when the requirements document is produced.
- Each organisation has its own version of this general model, depending on local factors: staff expertise, the type of system and the standards used.
- **Viewpoints** help organise stakeholder information: a viewpoint collects the requirements of one group of stakeholders who have something in common (e.g. end-users, managers). Viewpoints can also represent domain requirements and constraints from other systems. Alternatively, a model of the system architecture can be used to link requirements to subsystems.
- Hold **regular stakeholder meetings**: stakeholders who feel their views were ignored may deliberately try to undermine the RE process.
- When documenting, use **simple language and diagrams**, and a shared document or wiki that all stakeholders can access.

Requirements discovery ★★★ MUST KNOW

> **DEFINITION — LEARN THIS LINE**
> **Requirements discovery** — the process of gathering information about the required and existing systems and distilling the user and system requirements from this information.

- Interaction is with system stakeholders, from managers to external regulators.
- Systems normally have a range of stakeholders.
- Stakeholder information is supplemented by knowledge of existing systems and their usage, and by documents of various kinds.

There are two fundamental approaches: **interviewing**, where you talk to people about what they do, and **observation or ethnography**, where you watch people doing their job to see what artifacts they use and how they use them. Use a **mix** of both; stories and scenarios then help to discuss the requirements you derive.

Interviewing ★★★ MUST KNOW

Formal or informal interviews with stakeholders are part of most RE processes. The RE team asks stakeholders about the system they use now and the system to be developed, and derives requirements from the answers.

Closed interviews

Based on a **pre-determined list of questions**: the stakeholder answers a predefined set of questions.

Open interviews

No predefined agenda: various issues are explored with stakeholders to develop a better understanding of their needs.

Effective interviewing

- Be **open-minded**, avoid pre-conceived ideas about the requirements, and be willing to listen to stakeholders — if a stakeholder suggests something surprising, be willing to change your mind about the system.

- **Prompt** the interviewee to get discussions going using a **springboard question**, a **requirements proposal**, or by working together on a **prototype** system. Saying "tell me what you want" rarely gives useful information; people talk more easily in a defined context.

Interviews in practice

- Normally a **mix of closed and open-ended** interviewing: some questions must be answered, and these lead to less structured discussion. Completely open-ended discussions rarely work well.
- Interviews are good for getting an **overall understanding** of what stakeholders do, how they might interact with the system, and the difficulties they face with current systems.
- Interviewers need to be open-minded, without pre-conceived ideas of what the system should do.
- You need to prompt the user to talk about the system by **suggesting requirements** rather than simply asking them what they want. Unless you have a prototype to show, don't expect specific, detailed requirements — you analyse the information and generate the requirements yourself.

Problems with interviews

- Application specialists may use **language (jargon)** to describe their work that isn't easy for the requirements engineer to understand.
- Interviews are **not good for understanding domain requirements**:
  - requirements engineers cannot understand specific domain terminology;
  - some domain knowledge is so familiar that people find it hard to articulate, or think it isn't worth articulating. (For a librarian, it goes without saying that all acquisitions are catalogued before being added to the library — so the interviewer never hears it.)
- Interviews are also poor for **organisational requirements and constraints**: there are subtle power relationships, and people are reluctant to reveal the real (not the published) structure of decision-making to a stranger.
- On its own, interviewing is liable to miss essential information, so use it together with other techniques.

Ethnography ★★★ MUST KNOW

Software systems are used in a social and organisational environment. One reason why many systems are delivered but never used is that their requirements do not take proper account of how social and organisational factors affect the practical operation of the system.

> **DEFINITION — LEARN THIS LINE**
>
> **Ethnography** — an observational technique in which a social scientist spends a considerable time observing and analysing how people actually work, immersed in the working environment, to derive requirements for software that supports that work.

- People **do not have to explain or articulate** their work — which is useful, because their work is often "second nature" to them.
- **Social and organisational factors** of importance may be observed. (E.g. a workgroup may self-organise so that members can cover for each other — something never mentioned in an interview.)
- Ethnographic studies have shown that work is usually **richer and more complex than suggested by simple system models**. Suchman's pioneering study of office work found real practice far richer than office-automation models assumed — the main reason those systems had no significant effect on productivity.
- Its value is that it discovers **implicit** requirements that reflect how people actually work, not the formal processes defined by the organisation.

Scope of ethnography

Ethnography is particularly effective for discovering two types of requirements:

1. Requirements derived from the way that people **actually work**, rather than the way in which process definitions suggest that they ought to work. *Example:* air traffic controllers may switch off a conflict-alert system that sounds warnings even when planes are far apart, and use other strategies to keep planes on safe paths.
2. Requirements derived from **cooperation and awareness of other people's activities** — awareness of what other people are doing leads to changes in the ways in which we do things. *Example:* controllers watch adjacent sectors to predict how many aircraft will enter their own sector, so an automated ATC system should show some of the work in adjacent sectors.

Limitation: ethnography is effective for understanding **existing processes** but **cannot identify new features** that should be added to a system. Commentators suggest Nokia used ethnography to study how people used their phones, while Apple ignored current use and revolutionised the industry with the iPhone. Because it focuses on the end-user, ethnography is also weak for broader organisational or domain requirements — so it is one technique among several.

Focused ethnography

- Developed in a project studying the **air traffic control** process.
- **Combines ethnography with prototyping**.
- Prototype development results in **unanswered questions**, which focus the ethnographic analysis.
- The problem with ethnography is that it studies **existing practices**, which may have some historical basis that is no longer relevant.

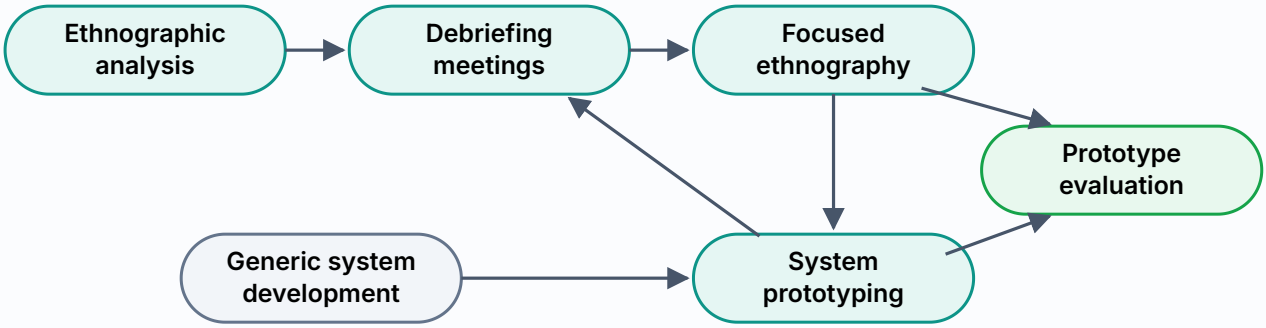


Fig 4.9 Ethnography and prototyping for requirements analysis. Sommerville Fig 4.8 · Your slides, slide 44

The ethnography **informs** the development of the prototype, so fewer prototype refinement cycles are needed. The prototyping **focuses** the ethnography by identifying problems and questions, which are discussed with the ethnographer in debriefing meetings; the ethnographer then looks for answers in the next phase of the study.

Basis	Interviewing	Ethnography
Method	Talk to people about what they do	Watch people doing their job
Who explains the work?	The stakeholder must put it into words	Nobody needs to; the observer records it

Basis	Interviewing	Ethnography
Good for	Overall understanding of tasks, how people would use the system, problems with current systems	Real working practice; cooperation and awareness; social and organisational factors
Weak for	Domain requirements (jargon, tacit knowledge); organisational and political issues	New features (innovation); broader organisational and domain requirements
Time needed	Relatively short sessions	Considerable time in the workplace
Typical risk	Interviewer's pre-conceived ideas; missing "obvious" knowledge	Copying existing practices whose historical basis is no longer relevant

Stories and scenarios ★★★ MUST KNOW

- Scenarios and user stories are **real-life examples** of how a system can be used.
- Stories and scenarios are a description of how a system may be used for a **particular task** — what people do, what information they use and produce, and what systems they may use.
- Because they are based on a practical situation, stakeholders can **relate to them** and can comment on their situation with respect to the story.

Story vs scenario: both describe system use for a task; they differ in structure and level of detail. A **story** is narrative text giving a high-level, "big picture" description. A **scenario** is a structured form of user story with specific information such as inputs and outputs. Parts of a story are developed into scenarios.

> **STORY — PHOTO SHARING IN THE CLASSROOM (ILEARN)**
>
> Jack is a primary school teacher in Ullapool (a village in northern Scotland). He has decided that a class project should be focused around the fishing industry in the area, looking at the history, development and economic impact of fishing. As part of this, pupils are asked to gather and share reminiscences from relatives, use newspaper archives and collect old photographs related to fishing and fishing communities in the area. Pupils use an iLearn wiki to gather together fishing stories and SCRAN (a history resources site) to access newspaper archives and photographs. However, Jack also needs a photo sharing site as he wants pupils to take and comment on each other's photos and to upload scans of old photographs that they may have in their families.
>
> Jack sends an email to a primary school teachers group, which he is a member of, to see if anyone can recommend an appropriate system. Two teachers reply and both suggest that he uses KidsTakePics, a photo sharing site that allows teachers to check and moderate content. As KidsTakePics is not integrated with the iLearn authentication service, he sets up a teacher and a class account. He uses the iLearn setup service to add KidsTakePics to the services seen by the pupils in his class so that when they log in, they can immediately use the system to upload photos from their mobile devices and class computers.

The advantage of stories is that **everyone can relate to them**. The iLearn team put such stories on a wiki and invited teachers and students across the country to comment — reaching a wider community than they could ever interview. User stories in agile methods such as Extreme Programming are really narrative scenarios.

What a scenario should include

A **scenario** is a structured form of user story. It starts with an outline of the interaction, and details are added during elicitation. Scenarios should include:

1. A description of the **starting situation** (what the system and users expect when the scenario starts);
2. a description of the **normal flow of events**;
3. a description of **what can go wrong** (and how the resulting problems are handled);
4. information about **other concurrent activities**;
5. a description of the **state when the scenario finishes**.

> **MEMORY TRICK**
> **S-N-W-O-E** — "*Start, Normal, Wrong, Others, End*". Think of a short film: opening scene, normal plot, what goes wrong, side plots, final scene.

Scenario part	Uploading photos to KidsTakePics (iLearn)
Initial assumption	A user or a group of users have one or more digital photographs to be uploaded to the picture sharing site. These are saved on either a tablet or laptop computer. They have successfully logged on to KidsTakePics.
Normal	The user chooses upload photos and is prompted to select the photos to be uploaded on their computer and to select the project name under which the photos will be stored. They should also be given the option of inputting keywords that should be associated with each uploaded photo. Uploaded photos are named by creating a conjunction of the user name with the filename of the photo on the local computer. On completion of the upload, the system automatically sends an email to the project moderator asking them to check new content, and generates an on-screen message to the user that this has been done.
What can go wrong	No moderator is associated with the selected project. An email is automatically generated to the school administrator asking them to nominate a project moderator. Users should be informed that there could be a delay in making their photos visible. Photos with the same name have already been uploaded by the same user. The user should be asked if they wish to re-upload the photos with the same name, rename the photos or cancel the upload. If they choose to re-upload the photos, the originals are overwritten. If they choose to rename the photos, a new name is automatically generated by adding a number to the existing file name.
Other activities	The moderator may be logged on to the system and may approve photos as they are uploaded.
System state on completion	User is logged on. The selected photos have been uploaded and assigned a status "awaiting moderation". Photos are visible to the moderator and to the user who uploaded them.

The key feature that makes KidsTakePics different from other photo sites is that a **teacher moderates** uploaded photos before they are shared. The scenario is much more detailed than the story, so it can be used to propose specific requirements.

Basis	Story	Scenario	Use case (Section 4.5)
Form	Narrative text	Structured text under fixed headings	UML diagram + textual description
Detail	High level, "big picture"	Detailed: inputs, outputs, exceptions	Names actors and each type of interaction
Best use	Start discussion with many stakeholders	Propose specific requirements	Describe all interactions; also used in design

Basis	Story	Scenario	Use case (Section 4.5)
Example	Jack's photo-sharing story	Uploading photos to KidsTakePics	Mentcare use cases (Fig 4.10)

4.5 Requirements specification ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Requirements specification — the process of writing down the user and system requirements in a requirements document.

- **User requirements** have to be understandable by end-users and customers who do not have a technical background. Write them in natural language with simple tables, forms and intuitive diagrams; avoid software jargon and structured or formal notations; describe only external behaviour, not architecture or design.
- **System requirements** are more detailed requirements and may include more technical information. They are the starting point for system design.
- The requirements may be part of a **contract** for the system development, so it is important that they are as **complete as possible**.
- Ideally requirements are clear, unambiguous, easy to understand, complete and consistent. In practice this is almost impossible, because stakeholders interpret requirements differently and there are inherent conflicts.

Ways of writing a system requirements specification ★★★ MUST KNOW

Notation	Description
Natural language	The requirements are written using numbered sentences in natural language. Each sentence should express one requirement.
Structured natural language	The requirements are written in natural language on a standard form or template. Each field provides information about an aspect of the requirement.
Design description languages	This approach uses a language like a programming language, but with more abstract features, to specify the requirements by defining an operational model of the system. This approach is now rarely used although it can be useful for interface specifications.
Graphical notations	Graphical models, supplemented by text annotations, are used to define the functional requirements for the system; UML use case and sequence diagrams are commonly used.
Mathematical specifications	These notations are based on mathematical concepts such as finite-state machines or sets. Although these unambiguous specifications can reduce the ambiguity in a requirements document, most customers don't understand a formal specification. They cannot check that it represents what they want and are reluctant to accept it as a system contract.

MEMORY TRICK

N-S-D-G-M — "*Nice Students Draw Good Maps*": Natural language, Structured natural language, Design description languages, Graphical notations, Mathematical specifications.

Requirements and design ★★ IMPORTANT

In principle, requirements should state **what** the system should do, and the design should describe **how** it does this. In practice, requirements and design are **inseparable**:

- A **system architecture may be designed to structure the requirements** — the requirements are organised by subsystem (as was done for iLearn).
- The system may **inter-operate with other systems** that generate design requirements (existing systems constrain the design).
- The use of a **specific architecture to satisfy non-functional requirements** may be a domain requirement (e.g. N-version programming to achieve reliability).
- This may be the consequence of a **regulatory requirement** — a regulator who must certify that the system is safe may insist on an architecture that has already been certified.

Natural language specification ★★★ MUST KNOW

- Requirements are written as natural language sentences supplemented by diagrams and tables.
- Natural language is used because it is **expressive, intuitive and universal**, so the requirements can be understood by users and customers. It has been used since the 1950s and is still the most widely used way of specifying requirements.
- But it is also potentially **vague and ambiguous**, and its meaning depends on the background of the reader.

Guidelines for writing requirements

1. **Invent a standard format** and use it for all requirements. Standardising makes omissions less likely and requirements easier to check (ideally one or two sentences per requirement).
2. **Use language in a consistent way.** Use **shall** for mandatory requirements and **should** for desirable requirements.
3. **Use text highlighting** (bold, italic or colour) to identify key parts of the requirement.
4. **Avoid the use of computer jargon**, abbreviations and acronyms — words like "architecture" and "module" are easily misunderstood.
5. **Include an explanation (rationale)** of why a requirement is necessary, and who proposed it (its source), so you know whom to consult if it has to change.

Problems with natural language

Problem	Meaning
Lack of clarity	Precision is difficult without making the document difficult to read.
Requirements confusion	Functional and non-functional requirements tend to be mixed up.
Requirements amalgamation	Several different requirements may be expressed together (in one sentence).

EXAMPLE — INSULIN PUMP REQUIREMENTS IN NATURAL LANGUAGE

3.2 The system **shall** measure the blood sugar and deliver insulin, if required, every 10 minutes. *(Changes in blood sugar are relatively slow so more frequent measurement is unnecessary; less frequent measurement could lead to unnecessarily high sugar levels.)*

3.6 The system **shall** run a self-test routine every minute with the conditions to be tested and the associated actions defined in Table 1. *(A self-test routine can discover hardware and software problems and alert the user to the fact the normal operation may be impossible.)*

Guidelines in action: a numbered standard format, "shall" for mandatory, the key word highlighted, and the rationale in brackets.

Structured specifications ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Structured specification — an approach to writing requirements where the freedom of the requirements writer is limited and requirements are written in a standard way (using templates/forms).

- Keeps most of the expressiveness and understandability of natural language, but imposes uniformity.
- This works well for some types of requirements, e.g. requirements for an **embedded control system**, but is sometimes **too rigid** for writing **business system** requirements.
- Templates may use programming-language constructs to show alternatives and iteration, and highlight key elements with shading or different fonts. The specification can be organised around the objects, functions or events of the system.
- (In the VOLERE method, each user requirement is first written on a card with fields such as rationale, dependencies on other requirements, source and supporting materials.)

Form-based specifications — the information on the form

1. Definition of the **function or entity**.
2. Description of **inputs** and where they come from.
3. Description of **outputs** and where they go to.
4. Information about the information needed for the computation and other entities used (the **"requires"** part).
5. Description of the **action** to be taken.
6. **Pre and post conditions** (if appropriate) — what must be true before the function is called, and what is true after it.
7. The **side effects** (if any) of the function.

Insulin Pump / Control Software / SRS / 3.3.2	
Function	Compute insulin dose: safe sugar level.
Description	Computes the dose of insulin to be delivered when the current measured sugar level is in the safe zone between 3 and 7 units.
Inputs	Current sugar reading (r2); the previous two readings (r0 and r1).
Source	Current sugar reading from sensor. Other readings from memory.
Outputs	CompDose — the dose in insulin to be delivered.

Insulin Pump / Control Software / SRS / 3.3.2	
Destination	Main control loop.
Action	CompDose is zero if the sugar level is stable or falling or if the level is increasing but the rate of increase is decreasing. If the level is increasing and the rate of increase is increasing, then CompDose is computed by dividing the difference between the current sugar level and the previous level by 4 and rounding the result. If the result is rounded to zero then CompDose is set to the minimum dose that can be delivered.
Requires	Two previous readings so that the rate of change of sugar level can be computed.
Pre-condition	The insulin reservoir contains at least the maximum allowed single dose of insulin.
Post-condition	r0 is replaced by r1 then r1 is replaced by r2.
Side effects	None.

A structured specification of a requirement for an insulin pump (Sommerville Fig 4.13; your slides, slides 60–61).

Structured specifications **reduce variability** and organise requirements more effectively. But it is still hard to write complex computations (such as the insulin dose) clearly — so add **tables** or graphical models.

Tabular specification ★★★ MUST KNOW

- Used to **supplement natural language**.
- Particularly useful when you have to define a number of possible **alternative courses of action**.
- For example, the insulin pump system bases its computations on the **rate of change of blood sugar level**, and the tabular specification explains how to calculate the insulin requirement for different scenarios. Here r0, r1 and r2 are the last three sugar readings, r2 being the current one.

Condition	Action
Sugar level falling (r2 < r1)	CompDose = 0
Sugar level stable (r2 = r1)	CompDose = 0
Sugar level increasing and rate of increase decreasing ((r2 − r1) < (r1 − r0))	CompDose = 0
Sugar level increasing and rate of increase stable or increasing ((r2 − r1) ≥ (r1 − r0))	CompDose = round ((r2 − r1)/4) If rounded result = 0 then CompDose = MinimumDose

? QUICK CHECK

Readings r0 = 4, r1 = 5, r2 = 6. What is CompDose?

Answer: The level is rising and the rate of increase is stable ((6 − 5) = 1 ≥ (5 − 4) = 1), so CompDose = round(1/4) = round(0.25) = 0; because the rounded result is 0, CompDose = MinimumDose.

Use cases ★★★ MUST KNOW

- Use cases are a kind of **scenario** that are included in the **UML** (they were first introduced in the Objectory method).
- Use cases identify the **actors** in an interaction and describe the **interaction** itself.
- A set of use cases should describe **all possible interactions** with the system.
- A **high-level graphical model** is supplemented by a more detailed **tabular description** (Chapter 5).

- UML **sequence diagrams** may be used to add detail to use cases by showing the sequence of event processing in the system.

Notation: actors — people or other systems — are drawn as **stick figures**; each class of interaction is a **named ellipse**; **lines** link actors with the interactions they take part in. Arrowheads may be added to show how an interaction is initiated.

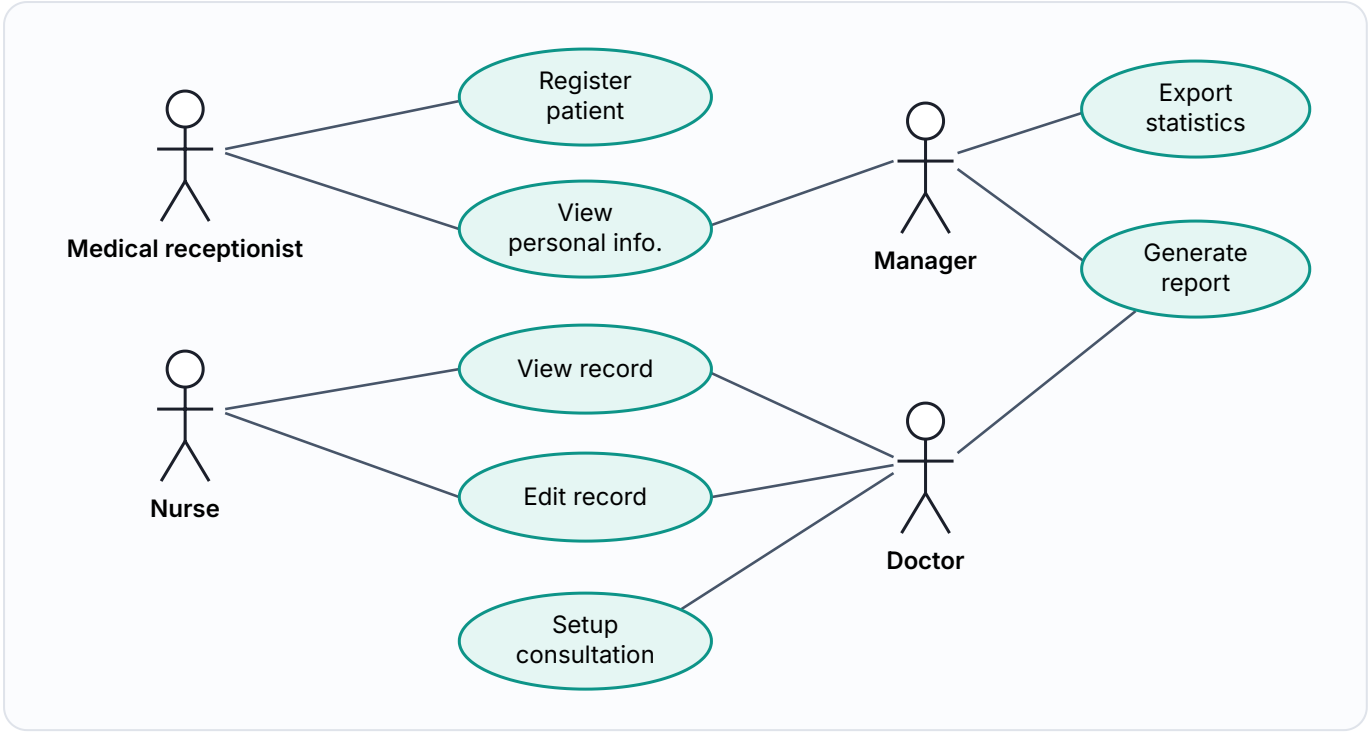


Fig 4.10 Use cases for the Mentcare system: four actors and seven use cases. Sommerville Fig 4.15 · Your slides, slide 65

EXAMPLE — TEXTUAL DESCRIPTION OF THE SETUP CONSULTATION USE CASE

Setup consultation allows two or more doctors, working in different offices, to view the same patient record at the same time. One doctor initiates the consultation by choosing the people involved from a dropdown menu of doctors who are online. The patient record is then displayed on their screens, but only the initiating doctor can edit the record. In addition, a text chat window is created to help coordinate actions. It is assumed that a phone call for voice communication can be separately arranged.

Sommerville's view: stakeholders often don't understand the term "use case", don't find the diagram useful and are not interested in every detailed interaction — so use cases are often **more helpful in system design** than in RE. Some people treat each use case as a single low-level scenario; others (Stevens and Pooley) treat a use case as a set of related scenarios — one for the normal interaction plus one for each possible exception.

The software requirements document ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Software requirements document (also called the software requirements specification, **SRS**) — the official statement of what is required of the system developers. It should include both a definition of user requirements and a specification of the system requirements.

- It is an **agreed statement** of the system requirements, organised so that both system customers and software developers can use it.
- It is **NOT a design document**. As far as possible, it should set out **WHAT** the system should do rather than **HOW** it should do it.
- It is essential when systems are **outsourced**, when **different teams** develop different parts of the system, and when a **detailed analysis** of the requirements is mandatory. It may not be needed for software products or business systems.
- Agile methods argue that it is out of date as soon as it is written; they collect user requirements incrementally as short user stories.

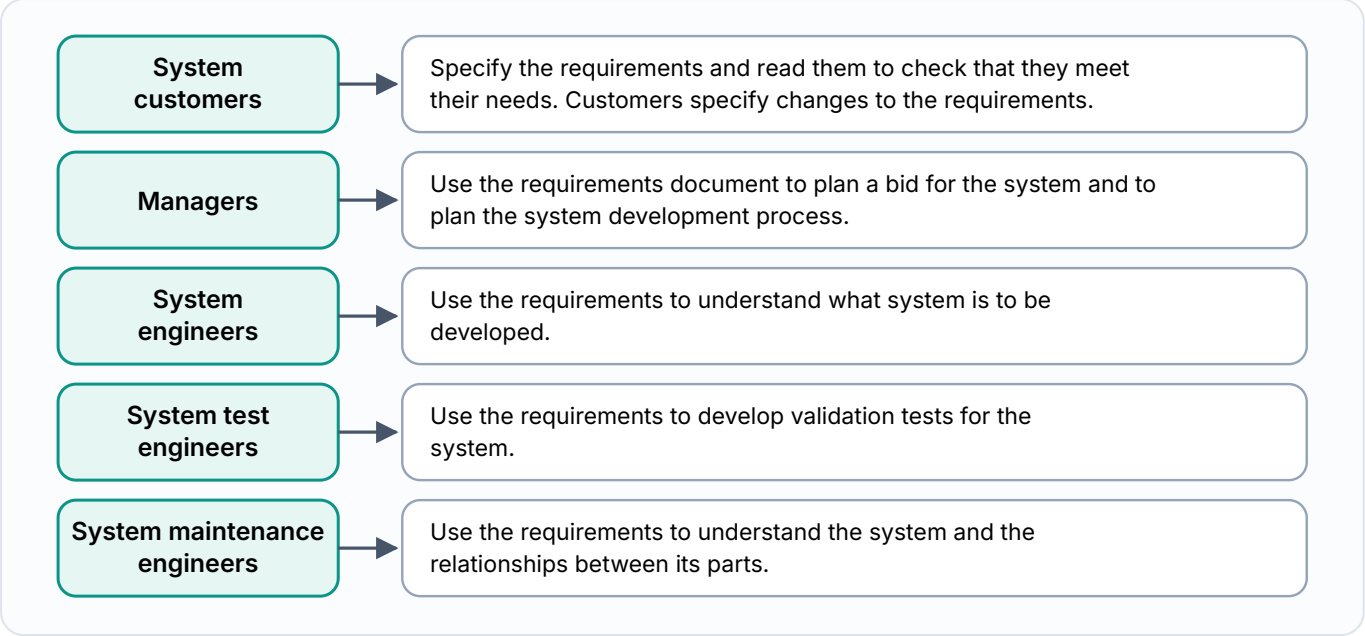


Fig 4.11 Users of a requirements document and how they use it. Sommerville Fig 4.16 · Your slides, slide 67

Because its users are so diverse, the document is a **compromise**: it must describe requirements for customers, define them in precise detail for developers and testers, and include information about likely future evolution (which helps designers avoid restrictive decisions and helps maintenance engineers adapt the system).

Requirements document variability

- Information in the requirements document depends on the **type of system** and the **approach to development** used.
- Systems developed **incrementally** will, typically, have **less detail** in the requirements document (with in-house iterative development, details are added during development).
- **Critical systems** need detailed requirements, because safety and security must be analysed; **outsourced** systems need detailed and precise specifications.
- Requirements document **standards** have been designed, e.g. the **IEEE standard**. These are mostly applicable to the requirements for large systems engineering projects — such as military command-and-control systems with a long lifetime, developed by a group of organisations.

The structure of a requirements document

One possible organisation, based on the IEEE standard and extended with information about predicted system evolution:

Chapter	Description
Preface	Defines the expected readership of the document and describes its version history, including a rationale for the creation of a new version and a summary of the changes made in each version.
Introduction	Describes the need for the system. It should briefly describe the system's functions and explain how it will work with other systems. It should also describe how the system fits into the overall business or strategic objectives of the organisation commissioning the software.
Glossary	Defines the technical terms used in the document. You should not make assumptions about the experience or expertise of the reader.
User requirements definition	Describes the services provided for the user. The non-functional system requirements should also be described in this section. This description may use natural language, diagrams, or other notations that are understandable to customers. Product and process standards that must be followed should be specified.
System architecture	Presents a high-level overview of the anticipated system architecture, showing the distribution of functions across system modules. Architectural components that are reused should be highlighted.
System requirements specification	Describes the functional and non-functional requirements in more detail. If necessary, further detail may also be added to the non-functional requirements. Interfaces to other systems may be defined.
System models	Graphical system models showing the relationships between the system components and between the system and its environment. Examples: object models, data-flow models, semantic data models.
System evolution	Describes the fundamental assumptions on which the system is based, and any anticipated changes due to hardware evolution, changing user needs, and so on. Useful for system designers, as it may help them avoid design decisions that would constrain likely future changes.
Appendices	Detailed, specific information related to the application being developed — for example, hardware and database descriptions. Hardware requirements define the minimal and optimal configurations for the system. Database requirements define the logical organisation of the data used by the system and the relationships between data.
Index	Several indexes to the document may be included. As well as a normal alphabetic index, there may be an index of diagrams, an index of functions, and so on.

MEMORY TRICK

"PIG-U, four S's, A-I": **P**reface, **I**ntroduction, **G**lossary, **U**ser requirements definition → **S**ystem architecture, **S**ystem requirements specification, **S**ystem models, **S**ystem evolution → **A**ppendices, **I**ndex.

A long document for a complex system built by several companies needs a full table of contents and index. A document for an in-house software product leaves out many of these chapters and focuses on the user requirements and high-level non-functional requirements.

4.6 Requirements validation ★★★ MUST KNOW

IN SIMPLE WORDS

Validation asks: "Have we written down the *right* system?" — before anyone starts building it. Catching a wrong requirement on paper is cheap; catching it after delivery is very expensive.

DEFINITION — LEARN THIS LINE

Requirements validation — the process of demonstrating that the requirements define the system that the customer really wants.

- It overlaps with elicitation and analysis, as it is concerned with finding problems with the requirements.
- Requirements error costs are high, so validation is very important: fixing a requirements error after delivery may cost up to 100 times the cost of fixing an implementation error.
- Why so expensive? A requirements change usually means the system design and implementation must also change, and the system must then be retested.

Requirements checking ★★★ MUST KNOW

Check	Question to ask	Textbook detail
Validity	Does the system provide the functions which best support the customer's needs?	Requirements reflect the real needs of users — which may have changed since they were elicited.
Consistency	Are there any requirements conflicts?	No contradictory constraints and no different descriptions of the same function.
Completeness	Are all functions required by the customer included?	The document defines all functions and constraints intended by the system user.
Realism	Can the requirements be implemented given available budget and technology?	Use knowledge of existing technology; also consider the budget and schedule.
Verifiability	Can the requirements be checked?	Requirements are written so that tests can show the delivered system meets each one — this reduces disputes between customer and contractor.

MEMORY TRICK

V-C-C-R-V — "*Very Careful Clients Request Verification*": Validity, Consistency, Completeness, Realism, Verifiability.

Requirements validation techniques ★★★ MUST KNOW

1. **Requirements reviews** — systematic manual analysis of the requirements by a team of reviewers, who check for errors and inconsistencies.
2. **Prototyping** — using an executable model of the system to check requirements (Chapter 2). End-users and customers experiment with it and feed back requirements changes.
3. **Test-case generation** — developing tests for requirements to check testability. If a test is difficult or impossible to design, the requirement will usually be hard to implement and should be reconsidered. Writing tests from the user requirements before any code is an integral part of test-driven

development.

Requirements reviews ★★★ MUST KNOW

- Regular reviews should be held **while the requirements definition is being formulated**.
- **Both client and contractor staff** should be involved in reviews.
- Reviews may be **formal** (with completed documents) or **informal**. Good communication between developers, customers and users can resolve problems at an early stage.
- In a review, people from the customer and the developer read the document in detail and check for errors, anomalies and inconsistencies; once these are recorded, customer and developer negotiate how to solve them.

Review checks

Review check	Question
Verifiability	Is the requirement realistically testable?
Comprehensibility	Is the requirement properly understood?
Traceability	Is the origin of the requirement clearly stated?
Adaptability	Can the requirement be changed without a large impact on other requirements?

⚠ WATCH OUT

Don't mix the two lists. **Requirements checks** (5): validity, consistency, completeness, realism, verifiability. **Review checks** (4): verifiability, comprehensibility, traceability, adaptability. Only **verifiability** appears in both.

Be realistic about validation. It is hard to show that requirements meet a user's needs: users must picture the system in operation and imagine how it fits their work — hard even for skilled professionals. You rarely find all problems during validation, so further changes to correct omissions and misunderstandings are inevitable after the document has been agreed.

4.7 Requirements change ★★★ MUST KNOW

The requirements for large software systems are **always changing**. One reason is that these systems often address **"wicked" problems** — problems that cannot be completely defined — so the requirements are bound to be incomplete. During development, the stakeholders' understanding of the problem keeps changing, and the requirements must evolve to reflect it (Fig 4.12).

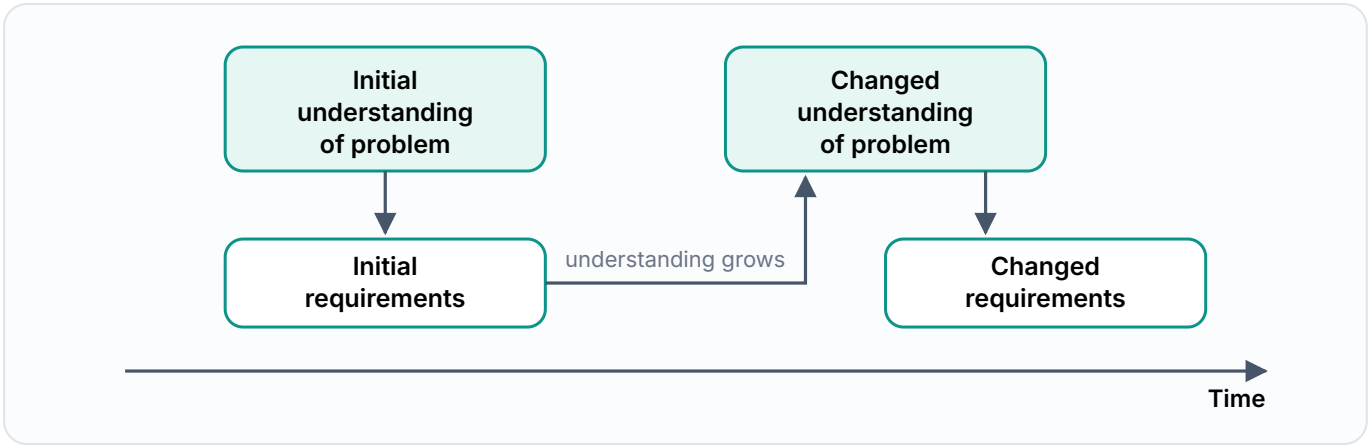


Fig 4.12 Requirements evolution: as understanding of the problem changes, the requirements change. Sommerville Fig 4.18 · Your slides, slide 80

Why requirements change ★★★ MUST KNOW

Once a system is installed and regularly used, new requirements inevitably emerge. Some come from errors and omissions in the original requirements, but most come from changes to the system's environment:

1. **The business and technical environment of the system always changes after installation.** New hardware may be introduced, it may be necessary to interface the system with other systems, business priorities may change (with consequent changes in the system support required), and new legislation and regulations may be introduced that the system must necessarily abide by.
2. **The people who pay for a system and the users of that system are rarely the same people.** System customers impose requirements because of organisational and budgetary constraints. These may conflict with end-user requirements and, after delivery, new features may have to be added for user support if the system is to meet its goals.
3. **Large systems usually have a diverse user community**, with many users having different requirements and priorities that may be conflicting or contradictory. The final system requirements are inevitably a compromise between them and, with experience, it is often discovered that the balance of support given to different users has to be changed.

Enduring and volatile requirements ★★ IMPORTANT

Basis	Enduring requirements	Volatile requirements
Meaning	Associated with the core, slow-to-change activities of an organisation — its fundamental work activities	More likely to change; associated with supporting activities that reflect *how* the organisation does its work rather than the work itself
Illustration (Mentcare)	Recording patients and the treatments they receive — a clinic always needs this	The layout of a management report, or the way staff log in (e.g. moving to ID cards)

Requirements management ★★★ MUST KNOW

> **DEFINITION — LEARN THIS LINE**
>
> **Requirements management** — the process of managing changing requirements during the requirements engineering process and system development.

- New requirements emerge as a system is being developed and after it has gone into use.

- You need to **keep track of individual requirements** and maintain **links between dependent requirements**, so that you can assess the impact of requirements changes.
- You need to establish a **formal process for making change proposals** and linking these to system requirements. Requirements management should start as soon as a draft of the requirements document is available.
- **Agile** processes handle change without a formal process: the user prioritises the change and, if it is high priority, decides which planned features to drop. The problem: users are not necessarily the best judges of whether a change is cost-effective. With many stakeholders, an **independent authority** that balances everyone's needs is often better.

Requirements management planning ★★★ MUST KNOW

Planning establishes the **level of requirements management detail** that is required. The requirements management decisions are:

Decision	What it means
Requirements identification	Each requirement must be uniquely identified so that it can be cross-referenced with other requirements (and used in traceability assessments).
A change management process	The set of activities that assess the impact and cost of changes (Fig 4.13).
Traceability policies	Define the relationships between each requirement, and between the requirements and the system design, that should be recorded — and how these records are maintained.
Tool support	Tools may range from specialist requirements management systems to spreadsheets and simple database systems.

Tool support is needed for **requirements storage** (a secure, managed data store accessible to everyone involved), **change management** (tracking suggested changes and the responses to them) and **traceability management** (discovering related requirements). Small systems can manage with shared web documents, spreadsheets and databases; large systems use specialised tools such as **DOORS**.

> **TRACEABILITY IN SIMPLE WORDS**
>
> Traceability is the "family tree" of each requirement: where it came from (its source), which other requirements depend on it, and which parts of the design implement it. When one requirement changes, you can follow the links to see how the change ripples through the system.

Requirements change management ★★★ MUST KNOW

Change management is applied to all proposed changes after the requirements document has been approved. It decides **whether a requirements change should be accepted** — whether the benefits of the change justify its cost. A formal process means all change proposals are treated consistently and the document is changed in a controlled way.

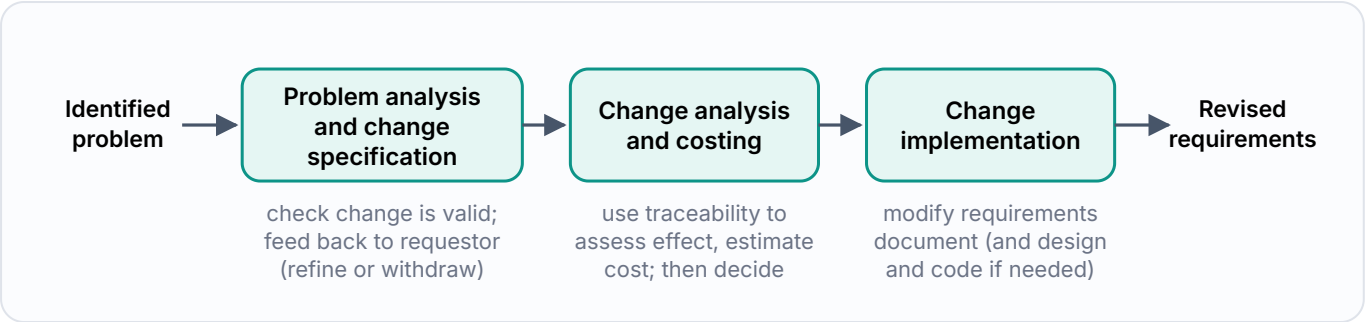


Fig 4.13 Requirements change management. Sommerville Fig 4.19 · Your slides, slides 83–84

1. **Problem analysis and change specification** — the process starts with an identified requirements problem or a specific change proposal. During this stage, the problem or the change proposal is analysed to check that it is valid. This analysis is fed back to the change requestor, who may respond with a more specific requirements change proposal, or decide to withdraw the request.
2. **Change analysis and costing** — the effect of the proposed change is assessed using traceability information and general knowledge of the system requirements. The cost is estimated in terms of changes to the requirements document and, if appropriate, to the design and implementation. Once this analysis is completed, a decision is made whether or not to proceed with the requirements change.
3. **Change implementation** — the requirements document and, where necessary, the system design and implementation are modified. Ideally, the document should be organised so that changes can be easily implemented (modular sections, few external references).

> ⚠ **WATCH OUT**
>
> Emergency changes: there is always a temptation to change the system first and update the requirements document later. The document and the implementation then get out of step, because changes are easily forgotten. If an emergency change is unavoidable, update the requirements document as soon as possible.

4.8 Lab: analysing a real-time case study ★★ IMPORTANT

The lab asks you to analyse the requirements of a **real-time** application — software that must react to its inputs within fixed time limits. The textbook's **insulin pump** is a good model: an embedded system that collects information from a sensor and controls a pump that delivers a controlled dose of insulin. It is **safety-critical**: if it fails, the user's health may be damaged. Use the same steps for any case study (weather station, traffic signal, ATM, ...).

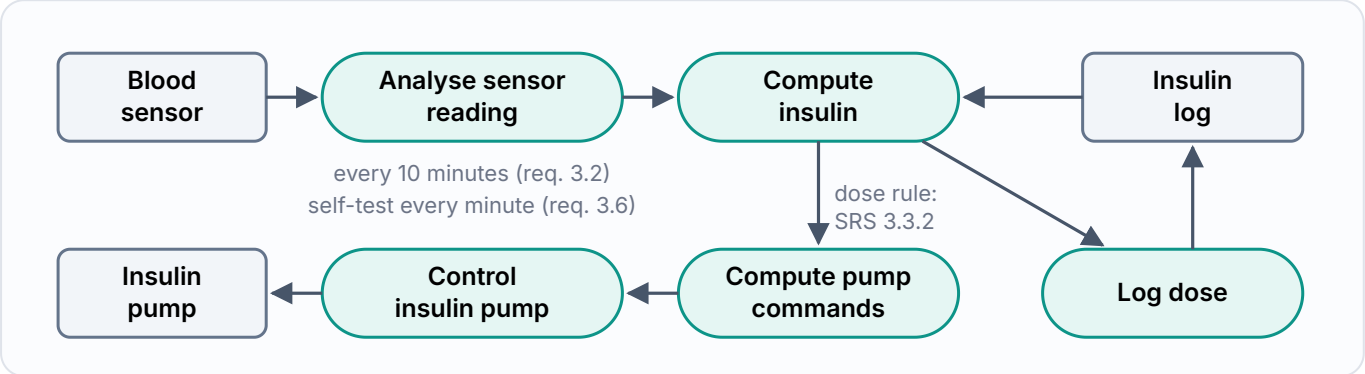


Fig 4.14 What the insulin pump software does, with the textbook requirements that govern each step. Simplified from Sommerville Fig 1.5 (activity model of the insulin pump)

1. **Understand the system and its environment** — read the case; sketch the hardware and the flow of data (Fig 4.14). Note the timing constraints.
2. **Identify the stakeholders** and the type of each (end users, managers, owners, external).
3. **Choose elicitation techniques** — interviews, observation, stories/scenarios — and say why.
4. **Write user requirements** — high-level, in natural language.
5. **Write functional system requirements** — numbered, "shall", with a rationale.
6. **Write non-functional requirements** — classify each (product / organisational / external) and make it measurable (metrics table).
7. **Specify one critical function** as a structured form or table; add a scenario or use-case diagram.
8. **Validate and plan for change** — apply the five checks; give every requirement an ID; mark enduring vs volatile ones.

Step	Insulin pump — model answer
System	Embedded, real-time, safety-critical control system: reads a blood-sugar sensor, computes the insulin needed and sends signals to a miniaturised pump (one pulse = one unit of insulin). Hardware: sensor, controller, pump, insulin reservoir, needle assembly, clock, alarm, two displays, power supply.
Stakeholders ***(illustrative)***	Patients with diabetes (end users); doctors who set up treatment; the manufacturer's engineers who maintain the device; medical-device regulators (external).
Elicitation ***(illustrative)***	Interviews with doctors and patients; observing how patients currently measure blood sugar and inject; scenarios such as "sugar level rising quickly after a meal".
User requirements (textbook)	(1) The system shall be available to deliver insulin when required. (2) The system shall perform reliably and deliver the correct amount of insulin to counteract the current level of blood sugar.
Functional requirements (textbook)	3.2 measure blood sugar and deliver insulin, if required, every 10 minutes · 3.6 run a self-test routine every minute · 3.3.2 compute insulin dose (structured form + table in Section 4.5).
Non-functional ***(illustrative wording)***	**Product (dependability):** user requirement (1) above is an availability requirement · **Product (performance):** each dose computation shall complete within the 10-minute measurement cycle · **Organisational (development):** the software shall be developed to the manufacturer's safety-critical process standard · **External (regulatory):** the pump shall be approved by the national medical-device regulator before use.
Validation	**Verifiability:** "every 10 minutes", "every minute" and the dose table are testable; "deliver the correct amount" becomes testable only through the table. **Realism:** can the controller hardware meet the timing? **Consistency:** the pre-condition (reservoir holds at least one maximum dose) must agree with the self-test and alarm requirements.
Change ***(illustrative)***	Enduring: the dose computation and safety requirements. Volatile: display layout, logging details. Give IDs (3.2, 3.3.2, 3.6…) so changes can be traced.

✎ EXAM TIP

Lab record layout: 1 Problem statement · 2 Stakeholders · 3 Elicitation method · 4 User requirements · 5 Functional requirements (numbered) · 6 NFRs (classified, with metrics) · 7 One structured or tabular specification · 8 Use-case diagram or scenario · 9 Validation checklist (V-C-C-R-V).

Chapter 4 in one page

Key definitions

- **RE**: establishing the services a customer requires and the constraints under which the system operates and is developed.
- **User reqs**: natural language + diagrams, for customers. **System reqs**: detailed, structured; may be part of the contract.
- **Stakeholder**: anyone affected by the system with a legitimate interest — end users, system managers, system owners, external.
- **FR**: services, reactions to inputs, behaviour in situations (and what not to do), or how computations are carried out. **NFR**: constraints on services — timing, process, standards; whole system. **Domain req**: constraint from the domain.
- **Goal** vs **verifiable NFR** (has a measure that can be objectively tested).
- **Validation**: show the requirements define the system the customer really wants.
- **Requirements management**: managing changing requirements during RE and development.

Lists to write in full

- **NFR tree**: Product (Usability, Efficiency [Performance, Space], Dependability, Security) · Organisational (Environmental, Operational, Development) · External (Regulatory, Ethical, Legislative [Accounting, Safety/security]).
- **Metrics**: speed, size, ease of use, reliability, robustness, portability.
- **Elicitation problems**: don't know what they want · own terms · conflicts · politics · change.
- **Scenario parts**: start, normal flow, what goes wrong, other activities, end state.
- **NL guidelines**: standard format · shall/should · highlighting · no jargon · rationale. **NL problems**: lack of clarity, confusion, amalgamation.

Processes

- **RE activities**: elicitation & analysis → specification → validation, plus management; iterative (spiral: business → user → system requirements).
- **Elicitation cycle**: discovery → classification & organisation → prioritisation & negotiation → specification.
- **Techniques**: interviews (closed/open) · ethnography (focused = + prototyping) · stories · scenarios · use cases.
- **Notations**: natural language, structured NL, design description languages, graphical, mathematical.
- **Form fields**: function, inputs+source, outputs+destination, requires, action, pre/post-conditions, side effects.
- **SRS chapters** (PIG-U-SSSS-AI): preface, introduction, glossary, user reqs, system architecture, system reqs spec, system models, system evolution, appendices, index.
- **Validation checks**: validity, consistency, completeness, realism, verifiability. **Techniques**: reviews, prototyping, test-case generation. **Review checks**: verifiability, comprehensibility, traceability, adaptability.
- **Why change**: environment changes · payers ≠ users · diverse users. **RM planning**: identification, change process, traceability policies, tool support. **Change mgmt**: problem analysis & change spec → change analysis & costing → change implementation.

Diagrams to practise

- Readers of requirements (4.2) · NFR tree (4.5) · spiral (4.7) · elicitation cycle (4.8) · ethnography + prototyping (4.9) · Mentcare use cases (4.10) · users of SRS (4.11) · evolution (4.12) · change management (4.13).

Likely exam questions

2M **Define requirements engineering. What is a stakeholder? Give two Mentcare stakeholders.**

How to answer → Slide definition of RE (services + constraints); stakeholder definition; e.g. doctors, medical receptionists (Section 4.1).

2M **Differentiate a goal and a verifiable non-functional requirement with an example.**

How to answer → Two definitions + the usability goal and its testable version (training hours, errors per hour).

4M **Distinguish between user requirements and system requirements. Who reads each?**

How to answer → Definitions → table → Mentcare drug-cost example (Fig 4.1) → readers diagram (Fig 4.2).

10M **Differentiate functional and non-functional requirements with suitable examples. Why can NFRs be more critical than FRs?**

How to answer → Definitions (also domain requirements) → comparison table (6+ rows) → Mentcare FR and NFR examples → aircraft/embedded examples of criticality → NFR implementation (architecture; generates FRs).

10M **With a neat diagram, explain the classification of non-functional requirements. Give an example of each main type.**

How to answer → Draw Fig 4.5 → define product, organisational, external with sub-types → Mentcare examples (availability, ID card, privacy standard).

5M **Explain the metrics used to specify non-functional requirements.**

How to answer → Six-row table: speed, size, ease of use, reliability, robustness, portability with their measures; why quantify (goals are not testable).

8M **Explain the spiral view of the requirements engineering process with a diagram.**

How to answer → List the RE activities → draw Fig 4.7 → early vs later effort, exit after user requirements, output = system requirements document.

8M **Explain the requirements elicitation and analysis process with a diagram. What problems arise during elicitation?**

How to answer → Fig 4.8 → four activities one line each → iterative → five problems (table in Section 4.4).

10M **Explain interviewing and ethnography as elicitation techniques. Compare them.**

How to answer → Closed/open interviews, effective interviewing, problems → ethnography definition, two types of requirements it finds, focused ethnography + Fig 4.9 → comparison table.

6M **What are stories and scenarios? What should a scenario include? Illustrate with the iLearn photo-upload scenario.**

How to answer → Story vs scenario → five parts (S-N-W-O-E) → fill them in for KidsTakePics.

6M **Explain the different notations for writing a system requirements specification.**

How to answer → Five-row table: natural language, structured NL, design description languages, graphical, mathematical — with one weakness each.

6M **Give the guidelines for writing natural language requirements. What problems does natural language cause?**

How to answer → Five guidelines (format, shall/should, highlight, no jargon, rationale) → three problems → insulin pump 3.2 example.

8M **Explain structured and tabular specification using the insulin pump example.**

How to answer → Definition, when suitable → seven form fields → the SRS 3.3.2 form → the four-row condition/action table.

6M **What are use cases? Draw the use case diagram for the Mentcare system (or for an ATM).**

How to answer → Five use-case points → notation (stick figures, ellipses, lines) → Fig 4.10 → one textual description (Setup consultation).

10M **Explain the structure of a software requirements document. Who uses it and how?**

How to answer → Definition (WHAT not HOW) → Fig 4.11 users → ten chapters with one line each → variability (IEEE standard, incremental = less detail).

8M **What is requirements validation? Explain the checks and techniques used, and the review checks.**

How to answer → Definition + 100× cost → five checks → three techniques → reviews → four review checks.

10M **Why do requirements change? Explain requirements management planning and the change management process with a diagram.**

How to answer → Wicked problems + Fig 4.12 → three reasons → four planning decisions + tool support → Fig 4.13 with three stages.

10M **For a real-time system (insulin pump or wilderness weather station), identify stakeholders and write functional and non-functional requirements.**

How to answer → Follow the eight lab steps in Section 4.8: stakeholders → user reqs → numbered "shall" FRs → classified, measurable NFRs → one structured spec → validation checks.

UNIT 2 · CHAPTER 5

Design Concepts

Design in software engineering · The design process · Design concepts · The design model

STUDY TIME
About 3.5 hours

SOURCES
Pressman 7th ed Ch 8 (no slides for this chapter) + notes from the 8th/9th ed

EXAM WEIGHT
High: design concepts, cohesion vs coupling and the design model are classic 8–10 mark questions

AFTER THIS CHAPTER YOU CAN

- Say where design sits and why it decides quality
- Draw how the requirements model becomes the design model
- List the quality guidelines and FURPS attributes
- Explain every design concept with an example
- Compare abstraction/refinement and cohesion/coupling
- Draw the modularity–cost graph
- Name the 5 design-class types and 4 well-formed traits
- Draw the design-model dimensions and explain each element

5.1 Design within the context of software engineering ★★★ MUST KNOW

What is software design? ★★★ MUST KNOW

IN SIMPLE WORDS

Requirements say **what** the system must do. Design decides **how** it will be built — its parts, how they connect, what data they keep, how users and other systems talk to it. It is the architect's *blueprint* that programmers follow.

Software design covers the set of **principles**, **concepts** and **practices** that lead to a high-quality system or product. Principles give the overall philosophy; concepts must be understood before design practice is applied; practice creates representations of the software that guide construction (coding and testing).

DEFINITION — LEARN THIS LINE

Design model — a representation of the software that gives detail about the **software architecture, data structures, interfaces and components** needed to implement the system. (The requirements model describes the required **data, function and behaviour**.)

Who / why? Software engineers do it. The design can be assessed for quality and improved *before* code is written and tested — design is where software quality is established.

Steps	Represent the **architecture** → model the **interfaces** (to users, other systems/devices, and between its own components) → design the **components**. All follow the basic design concepts.
Work product	A **design model** with architectural, interface, component-level and deployment representations.
Check	The team reviews it for errors, inconsistencies and omissions, better alternatives, and whether it can be built within constraints, schedule and cost.

Firmness, commodity and delight. Vitruvius said good buildings show these three; so should good software — **firmness**: no bugs that inhibit its function; **commodity**: suitable for the purpose intended; **delight**: pleasurable to use. Designers reach such a model by **diversification** (collecting a repertoire of alternatives — components, solutions, knowledge) and then **convergence** (choosing the elements that meet the requirements and rejecting the rest until one configuration remains).

Where design sits ★★★ MUST KNOW

Design is the **technical kernel** of software engineering and is applied **whatever process model** is used. It begins once requirements have been analysed and modelled; it is the last software engineering action within the modeling activity and sets the stage for **construction** (code generation and testing).

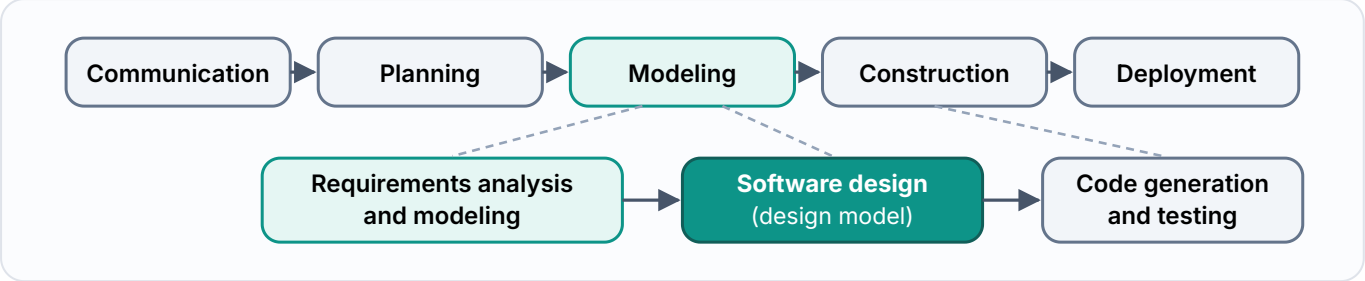


Fig 5.1 Design is the last action of modeling; its output, the design model, is the blueprint for construction (shown on Pressman's five framework activities). Drawn for this guide from Pressman 8.1

Translating the requirements model into the design model ★★★ MUST KNOW

The requirements model — its **scenario-based, class-based, flow-oriented and behavioural elements** — feeds the design task. Using design notation and methods, design produces the **four design models**: a **data/class design, an architectural design, an interface design and a component-level design**.

Design model	What it defines	Main input
Data/class design	Transforms class models into **design class realizations** and the **data structures** needed for implementation (partly with the architecture; more detail as each component is designed).	CRC objects and relationships; class attributes
Architectural design	The **relationships between major structural elements**, the **architectural styles and design patterns** that meet the requirements, and the **constraints** on implementation — the "framework" of the system.	The requirements model
Interface design	How the software **communicates** with interoperating systems and with the **humans** who use it. An interface implies a flow of information (data and/or control) and a specific behaviour.	Usage scenarios, behavioural models

Design model	What it defines	Main input
Component-level design	Transforms structural elements of the architecture into a **procedural description of software components**.	Class-based, flow and behavioural models

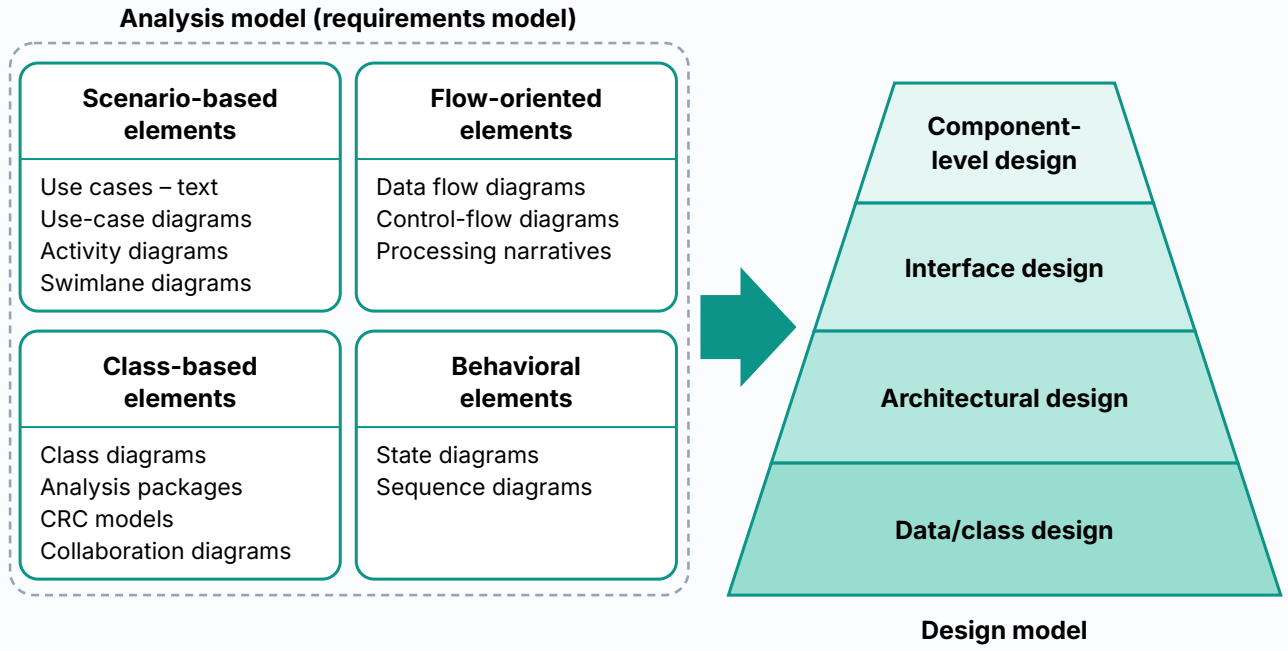


Fig 5.2 Translating the requirements model into the design model: the four kinds of requirements element feed the design task, which produces the four layers of the design model. Pressman Fig 8.1

★ KEY POINT

Design should **always begin with data** — the foundation for all other elements (the base of the pyramid). Then the **architecture** is derived; only then are the other design tasks done.

Why is design important? Quality ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Importance of design — in one word, **quality**: design is the place where **quality is fostered** in software engineering.

1. Design gives **representations of software that can be assessed for quality**.
2. It is the **only way to accurately translate stakeholders' requirements** into a finished product.
3. It is the **foundation for all the software engineering and support activities** that follow.
4. **Without design** the system may be unstable — it **fails when small changes are made**, is **difficult to test**, and its **quality cannot be assessed until late**, when time is short and money is spent.

Design vs coding (SafeHome): code is just the language used to *represent* a design. A programming language shows details (data structures, algorithms) well but not **architecture or component collaboration** — "a screwed-up architecture can ruin even the best code". The difference is the **level of abstraction**.

5.2 The design process ★★ IMPORTANT

Design is an **iterative process** that translates requirements into a **"blueprint"** for constructing the software. The first blueprint is *holistic* — at a **high level of abstraction**, traceable directly to the system objective and to the data, functional and behavioural requirements. Each iteration **refines** it to **lower levels of abstraction**, still traceable to requirements but more subtly.

Three characteristics of a good design ★★★ MUST KNOW

McGlaughlin's three characteristics guide the evaluation of a design (each is also a *goal* of the design process):

1. It must **implement all the explicit requirements** of the requirements model and **accommodate all the implicit requirements** desired by stakeholders.
2. It must be a **readable, understandable guide** for those who generate code and those who test and support the software.
3. It should give a **complete picture of the software**, addressing the data, functional and behavioural domains from an implementation perspective.

Quality guidelines ★★ IMPORTANT

1. A design should exhibit an **architecture** that (a) uses recognizable **architectural styles or patterns**, (b) is made of **components with good design characteristics**, and (c) can be implemented in an **evolutionary fashion** (small systems may be designed linearly).
2. A design should be **modular** — logically partitioned into elements or subsystems.
3. It should contain **distinct representations of data, architecture, interfaces and components**.
4. It should lead to **data structures** appropriate for the classes, drawn from recognizable data patterns.
5. It should lead to **components with independent functional characteristics**.
6. It should lead to **interfaces that reduce the complexity of connections** between components and with the environment.
7. It should be derived using a **repeatable method** driven by information from requirements analysis.
8. It should use a **notation that effectively communicates its meaning**.

These come from fundamental design principles, systematic methodology and thorough review. During design there is no code to test, so quality is checked by **technical reviews**: a meeting of two to four people — a **review leader** (plans and runs it), a **recorder** (takes notes) and the **producer** (whose work is reviewed) — who read the design beforehand and list errors, omissions and ambiguities (typically 90 minutes to 2 hours).

FURPS quality attributes ★★★ MUST KNOW

Hewlett-Packard's **FURPS** attributes are a **target for all software design**:

Attribute	Assessed / measured by
Functionality	The **feature set and capabilities** of the program, the **generality** of the functions delivered, and the **security** of the overall system.
Usability	**Human factors**, overall **aesthetics**, **consistency** and **documentation**.
Reliability	**Frequency and severity of failure**, **accuracy** of output, **mean-time-to-failure (MTTF)**, ability to **recover** from failure, and **predictability**.

Attribute	Assessed / measured by
Performance	**Processing speed, response time, resource consumption, throughput** and **efficiency**.
Supportability	**Extensibility, adaptability, serviceability** (together = *maintainability*) plus **testability, compatibility, configurability**, ease of **installation** and ease of **localizing problems**.

The attributes are **not weighted equally** (one system stresses functionality/security, another performance, another reliability), but all must be considered as design begins, not after construction starts.

> **MEMORY TRICK**
> **FURPS** — "**F**ast **U**sers **R**arely **P**ick **S**low apps". Supportability hides maintainability (extensibility + adaptability + serviceability).

Evolution of design and common features of design methods ★★ IMPORTANT

Evolution (about six decades): modular programs and top-down refinement → structured programming → translating data flow or data structure into a design → object-oriented design → software architecture and design patterns → aspect-oriented methods, model-driven and test-driven development.

All design methods share four characteristics: (1) a **mechanism for translating** the requirements model into a design representation; (2) a **notation** for functional components and their interfaces; (3) **heuristics for refinement and partitioning**; (4) **guidelines for quality assessment**.

Generic task set for design ★★ IMPORTANT

1. **Design data structures** for the data objects and attributes in the information domain model.
2. **Select an architectural style** appropriate for the software.
3. **Partition the analysis model into design subsystems** and allocate them in the architecture (each subsystem functionally cohesive; design subsystem interfaces; allocate analysis classes or functions).
4. **Create design classes or components**: translate analysis classes into design classes; check against design criteria and inheritance; define methods and messages; select design patterns; review and revise.
5. **Design interfaces** to external systems or devices.
6. **Design the user interface**: review task analysis; specify action sequences from scenarios; build a behavioural model; define interface objects and control mechanisms; review.
7. **Conduct component-level design**: specify algorithms at a low level of abstraction; refine each interface; define component data structures; review and correct errors.
8. **Develop a deployment model**.

5.3 Design concepts ★★★ MUST KNOW

A set of fundamental design concepts has **stood the test of time**; each gives the designer a foundation for more sophisticated methods, in both traditional and OO development. They help answer: **What criteria partition software into components? How is function or data-structure detail separated from a conceptual representation? What uniform criteria define the technical quality of a design?** As M. A. Jackson put it, wisdom begins with knowing the difference between **getting a program to work** and **getting it right**.

THE CONCEPTS IN TEXTBOOK ORDER + MEMORY TRICK

1 Abstraction · **2** Architecture · **3** Patterns · **4** Separation of concerns · **5** Modularity · **6** Information hiding · **7** Functional independence · **8** Refinement · **9** Aspects · **10** Refactoring · **11** OO design concepts · **12** Design classes · (newer editions: **13** Dependency inversion · **14** Design for test).

"**A**n **A**rchitect **P**lans **S**eparate **M**odules, **H**ides **I**nfo, keeps them **F**unctionally **I**ndependent, **R**efines, adds **A**spects, **R**efactors, thinks in **O**bjects and **D**esign classes."

1 · Abstraction ★★★ MUST KNOW

You say "book a train ticket" without listing the forty clicks behind it. A modular solution can be described at many **levels of abstraction**: at the **highest** level, in broad terms using the **language of the problem environment**; at **lower** levels, in more detail, mixing problem-oriented and **implementation-oriented terms**; at the **lowest** level, in a form that can be **directly implemented**.

DEFINITION — LEARN THIS LINE

Procedural abstraction — a sequence of instructions that have a **specific and limited function**. The **name** implies these functions, but specific **details are suppressed**.

Data abstraction — a **named collection of data that describes a data object**.

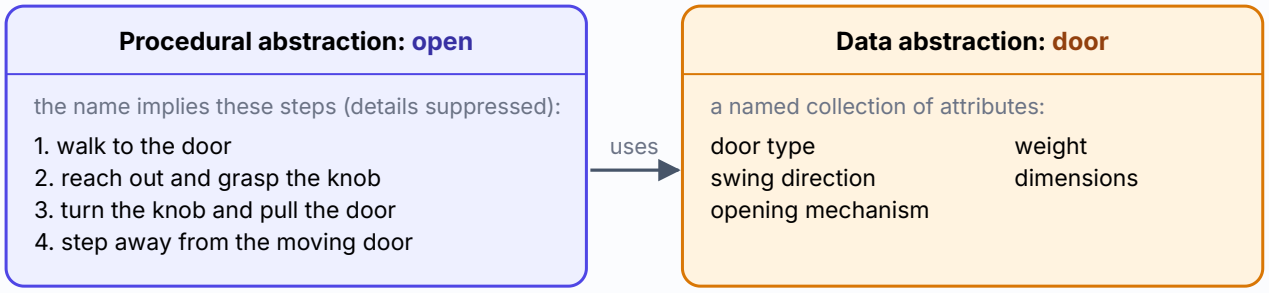


Fig 5.3 The textbook's door example: the procedural abstraction *open* uses the attributes of the data abstraction *door*. Drawn for this guide from Pressman 8.3.1

The steps behind *open* can be **replaced** (e.g., an automatic door with a sensor) as long as the function implied by the name stays the same — users of the abstraction are unaffected. SafeHome-style example: data abstraction **Sensor** (type, id, location, threshold) with procedural abstraction **armSystem()**.

✎ EXAM TIP

Open with Grady Booch: "Abstraction is one of the fundamental ways that we as humans cope with complexity." Then the levels, both types and the door example — a complete 5-mark answer.

2 · Architecture ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Software architecture — "the overall structure of the software and the ways in which that structure provides **conceptual integrity** for a system" (Shaw and Garlan).

In simplest form it is the **structure or organization of program components (modules)**, the **manner in which they interact**, and the **structure of data** they use (more broadly, major system elements and their interactions). Think of it as the software's **floor plan**. The architectural rendering is the **framework** for

detailed design; **architectural patterns** solve common design problems.

Property to specify (Shaw and Garlan)	Meaning
Structural properties	The **components** (modules, objects, filters) and how they are **packaged and interact** — e.g., objects encapsulate data and processing and interact by invoking methods.
Extra-functional properties	How the architecture achieves **performance, capacity, reliability, security, adaptability** and other characteristics.
Families of related systems	Draw on **repeatable patterns** found in families of similar systems — the ability to **reuse architectural building blocks**.

Five kinds of architectural model: **structural** (an organized collection of program components); **framework** (repeatable design frameworks of similar applications — higher abstraction); **dynamic** (behaviour — how structure or configuration changes with external events); **process** (the business or technical process the system must accommodate); **functional** (the functional hierarchy). **Architectural description languages (ADLs)** describe components and how they connect. There is debate whether architecture is derived *separately* (between requirements engineering and design) or is an *integral part* of design. Advice: don't let architecture "just happen" — design it explicitly.

> **MEMORY TRICK**
> Five models — "**S**ome **F**amous **D**esigners **P**lan **F**irst": Structural, Framework, Dynamic, Process, Functional.

3 · Patterns ★★ IMPORTANT

> **DEFINITION — LEARN THIS LINE**
> **Design pattern** — "a named nugget of insight which conveys the essence of a **proven solution** to a **recurring problem** within a certain **context** amidst competing concerns" (Brad Appleton).

A pattern describes a design structure that solves a particular problem in a specific context and amid **"forces"** that affect how it is applied. Its description lets the designer decide (1) whether it is **applicable** to the current work, (2) whether it can be **reused** (saving design time), and (3) whether it can **guide the development of a similar** but functionally or structurally different pattern. (Christopher Alexander: a pattern can be used "a million times over, without ever doing it the same way twice".) *Everyday example:* a traffic roundabout — one named, proven idea for crossing traffic, built differently each time. *Software:* the Observer pattern — when a SafeHome sensor trips, the display, alarm and dialler that depend on it are all notified.

4 · Separation of concerns ★★★ MUST KNOW

> **DEFINITION — LEARN THIS LINE**
> **Separation of concerns** — any complex problem can be more easily handled if it is **subdivided into pieces that can each be solved and/or optimized independently** (Dijkstra). A **concern** is a feature or behaviour specified in the requirements model.

If problem p1 is perceived as more complex than p2, it needs more effort. And the perceived complexity of two problems **combined** is often **greater than the sum** of each taken separately:

$$C(p1) > C(p2) \Rightarrow E(p1) > E(p2) \quad \text{and} \quad C(p1 + p2) > C(p1) + C(p2) \Rightarrow E(p1 + p2) > E(p1) + E(p2)$$

Hence **divide and conquer**. Separation of concerns appears in **modularity**, **aspects**, **functional independence** and **refinement**.

> ⚠ **WATCH OUT**
> It can be taken too far: split a problem into very many tiny pieces and each is easy, but **integration may be very difficult** — see the modularity graph (Fig 5.4).

5 · Modularity ★★★ MUST KNOW

> **DEFINITION — LEARN THIS LINE**
> **Modularity** — software is divided into **separately named and addressable components**, sometimes called **modules**, that are integrated to satisfy problem requirements. It is the most common manifestation of separation of concerns. "Modularity is the single attribute of software that allows a program to be **intellectually manageable**" (Myers).

Monolithic software (one large module) cannot be easily grasped — its number of **control paths**, **span of reference**, **number of variables** and overall **complexity** make understanding close to impossible. So the design is broken into modules — but not indefinitely:

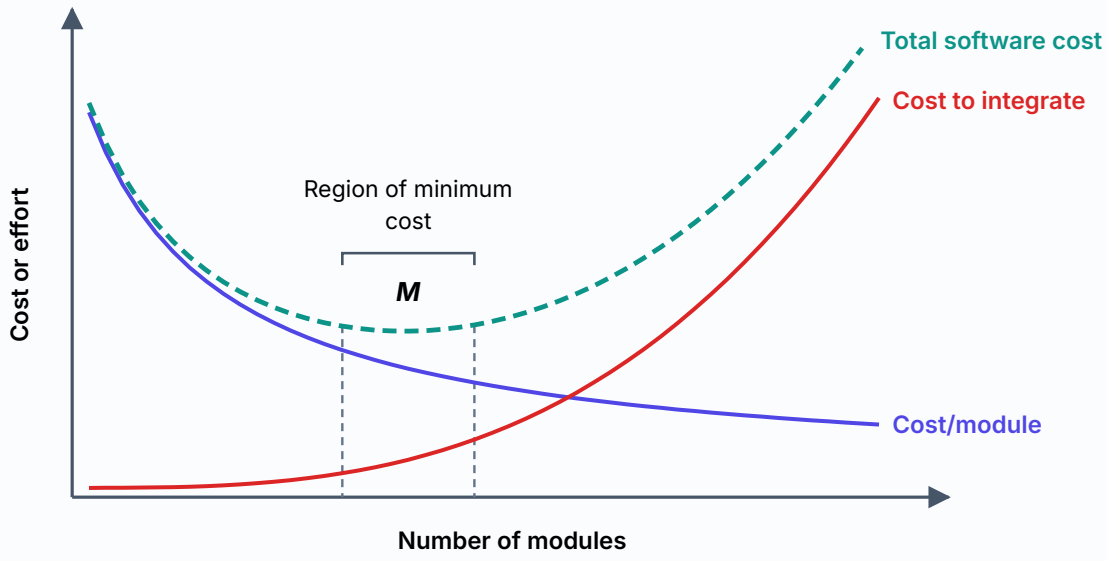


Fig 5.4 Modularity and software cost: cost per module falls as modules are added, the cost to integrate rises, and total cost is lowest near *M* modules. Pressman Fig 8.2

- **Cost/module** falls as the number of modules rises (same requirements, more modules ⇒ smaller modules); the **cost to integrate** rises.
- The **total cost** is U-shaped: some number ***M*** of modules gives minimum development cost, but we cannot predict *M* with assurance.
- So modularize, but stay in the **vicinity of *M*** — avoid undermodularity and overmodularity.

Why modularize? Development can be more easily **planned**; **increments** can be defined and delivered; **changes** are easier to accommodate; **testing and debugging** are more efficient; long-term **maintenance** has fewer serious side effects.

6 · Information hiding ★★★ MUST KNOW

Modularity raises the question "How do I decompose a solution to obtain the *best* set of modules?" Parnas answers it:

> **DEFINITION — LEARN THIS LINE**
>
> **Information hiding** — modules should be "characterized by design decisions that (each) hides from all others": the **information (algorithms and data) inside a module is inaccessible to other modules that have no need for it**.

Effective modularity comes from **independent modules that communicate only the information necessary** to achieve software function. **Abstraction** defines the procedural/informational entities; **hiding** defines and **enforces access constraints** to procedural detail and **local data structures**, which sit behind a **module interface**.

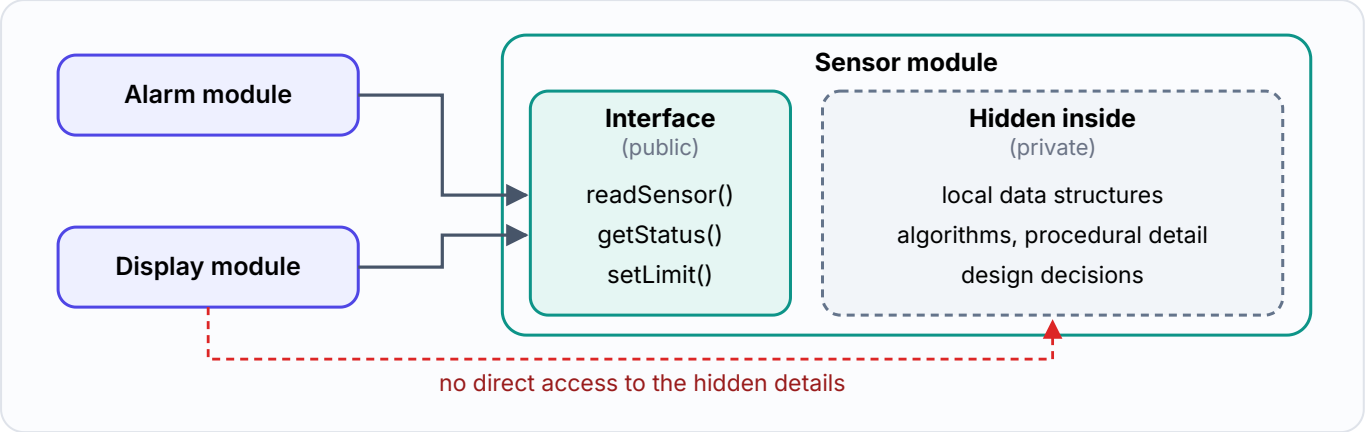


Fig 5.5 Information hiding: other modules use only the interface; data structures and algorithms stay hidden, so changing them cannot break the callers. Drawn for this guide

Benefit: greatest when **modifications are needed during testing and maintenance** — because most detail is hidden, errors introduced during modification are less likely to propagate to other parts of the software. (Like driving a car through its steering, pedals and gears: the engine can change from petrol to electric and you still drive the same way.)

7 · Functional independence: cohesion and coupling ★★★ MUST KNOW

Functional independence grows directly out of **separation of concerns, modularity, abstraction and information hiding** (Wirth, Parnas; solidified by Stevens, Myers and Constantine).

> **DEFINITION — LEARN THIS LINE**
>
> **Functional independence** — achieved by developing modules with **"single-minded" function** and an **"aversion" to excessive interaction** with other modules: each module addresses a **specific subset of requirements** and has a **simple interface**.
>
> **Cohesion** — an indication of the **relative functional strength** of a module (the degree to which it focuses on **just one thing**).
>
> **Coupling** — an indication of the **relative interdependence among modules** (the degree to which a module is connected to other modules and the outside world).

Why independence? Independent modules are **easier to develop** (function is compartmentalized, interfaces simplified — vital for teams) and **easier to maintain and test** (secondary effects of changes are limited, **error propagation is reduced**, and **reuse** becomes possible). "Functional independence is a key to good design, and design is the key to software quality."

- **Cohesion** extends information hiding: a cohesive module performs a **single task** needing little interaction with others — ideally it does **just one thing**. Multiple functions are sometimes necessary, but **"schizophrenic" modules** (many unrelated functions) must be avoided. Aim: high cohesion.
- **Coupling** depends on **interface complexity**, the **point at which entry or reference** is made to a module, and **what data pass across the interface**. Aim: lowest possible coupling — simple connectivity is easier to understand and less prone to the **"ripple effect"** (an error in one place propagating through the system).

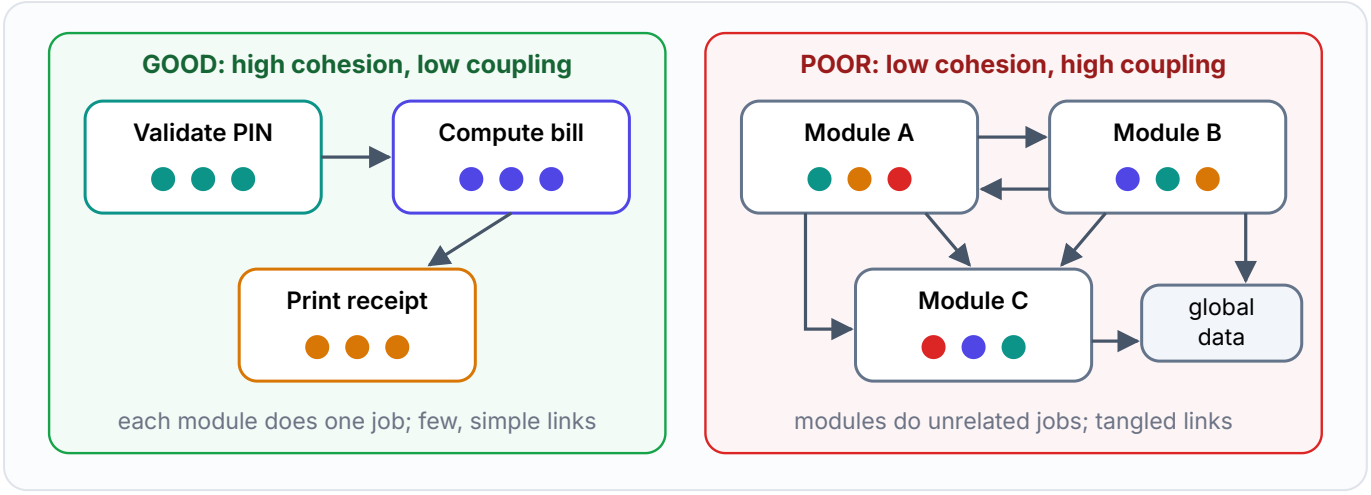


Fig 5.6 Functional independence. Dots of one colour = related functions. Left: single-minded modules with one simple link each. Right: mixed functions, many links and shared global data — a change ripples everywhere. Drawn for this guide

Point	Cohesion	Coupling
Measures	Functional strength **inside** one module	Interdependence **between** modules
Goal	**High** — the module does one thing	**Low** — few, simple connections
Depends on	How closely its functions and data are related	Interface complexity, point of entry/reference, data passed
Bad case	"Schizophrenic" module with unrelated jobs	"Ripple effect" of errors through the system

MEMORY TRICK

Co-HE-sion = inside one module; COUPLE-ing = between two. Good design = **high cohesion, low coupling** — "strong teams, few meetings".

Types of cohesion and coupling (Pressman Ch 10, component-level design)

Chapter 8 only defines the two ideas; their types are listed later in the book. Learn them if your paper asks for "types".

Cohesion type	Meaning (highest level first)
Functional	Shown mainly by operations: the component performs a targeted computation and returns a result.

Cohesion type	Meaning (highest level first)
Layer	Packages/components/classes: a higher layer uses services of a lower layer, but lower layers never access higher ones.
Communicational	All operations that access the same data are defined within one class.

Coupling categories: **content** (one component secretly modifies another's internal data — violates information hiding); **common** (components share a global variable); **control** (A() passes a control flag that directs B's logic); **stamp** (ClassB is the type of an argument of a ClassA operation); **data** (long strings of data arguments are passed); **routine call** (one operation invokes another); **type use** (A uses a data type defined in B); **inclusion/import** (A includes or imports B's package or content); **external** (communication with infrastructure — OS, database, telecom).

The classic scales (structured design — also widely examined)

Cohesion, worst → best: coincidental (unrelated parts) → logical (similar tasks, one chosen by a flag) → temporal (tasks done at the same time, e.g., start-up) → procedural (steps in a fixed order) → communicational (parts share the same data) → sequential (output of one part feeds the next) → functional (all parts serve one well-defined task).
Coupling, worst → best: content (uses/changes another's internals) → common (shared global data) → external (shared external format, protocol or device) → control (passes a control flag) → stamp (passes a whole data structure when only part is needed) → data (passes only the elementary data needed).

8 · Refinement (stepwise refinement) ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Stepwise refinement — a **top-down design strategy** proposed by **Niklaus Wirth**: a program is developed by **successively refining levels of procedural detail**; a macroscopic statement of function (a procedural abstraction) is decomposed step by step **until programming-language statements are reached**.

Refinement is a **process of elaboration**: start with a statement of function (or information) at a high level of abstraction that says nothing about internal workings, then add **more detail at each successive refinement**.

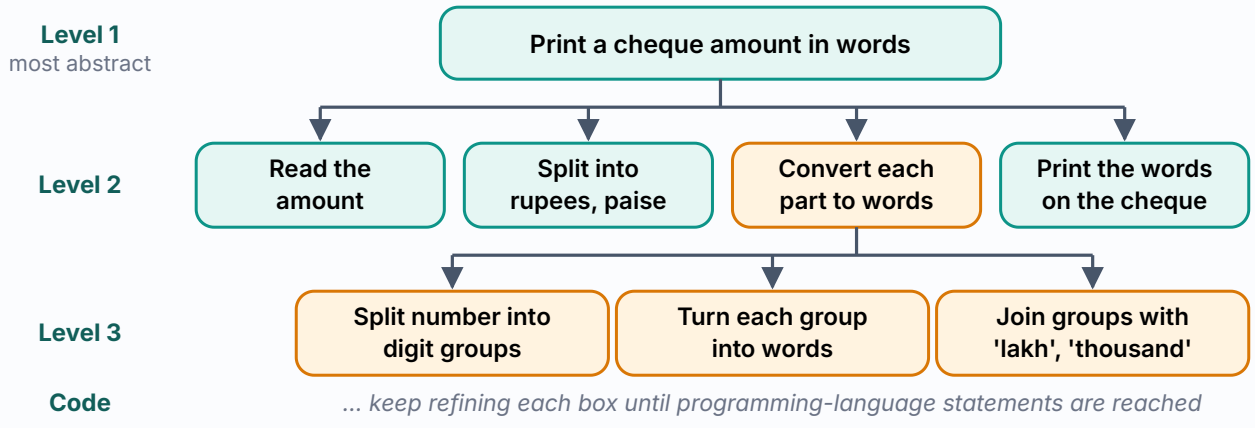


Fig 5.7 Stepwise refinement of a cheque writer (the textbook's exercise 8.12a): each level elaborates one box of the level above. Drawn for this guide

Abstraction and refinement are complementary. Abstraction lets you specify procedure and data internally but **suppress the need for "outsiders" to know low-level details**; refinement **reveals low-level details** as design progresses. Together they build the complete design model.

Point	Abstraction	Refinement
Direction	Hides detail — up to the big picture	Adds detail — down towards code
Purpose	Outsiders need not know low-level details	Reveals low-level details as design progresses
Result	Named procedural and data abstractions	A hierarchy of ever more detailed steps

⚠ **WATCH OUT**

Don't jump straight to full detail and skip refinement steps — it causes **errors and omissions** and makes the design much harder to review.

9 · Aspects ★★ IMPORTANT

Requirements analysis uncovers **concerns** — "requirements, use cases, features, data structures, quality-of-service issues, variants, intellectual property boundaries, collaborations, patterns and contracts". Ideally each is isolated, but **some concerns span the entire system** and cannot easily be compartmentalized.

DEFINITION — LEARN THIS LINE

Crosscutting — requirement A crosscuts requirement B "if a software decomposition [refinement] has been chosen in which B cannot be satisfied without taking A into account."

Aspect — a **representation of a crosscutting concern**.

Example (SafeHomeAssured.com): requirement **A** (use case ACS-DCV) lets a registered user view video from cameras placed in a space — design representation A*. Requirement **B**: a registered user must be **validated** before using SafeHomeAssured.com — it applies to *every* function; design representation B*. **B* crosscuts A***, so B* is an **aspect** of the WebApp.

Aspects must be identified so that the design can accommodate them during refinement and modularization. Ideally an aspect is implemented as a **separate module (component)**, not as fragments **"scattered" or "tangled"** through many components; the architecture should support a mechanism for defining such a module. Typical crosscutting concerns: security, logging, error handling.

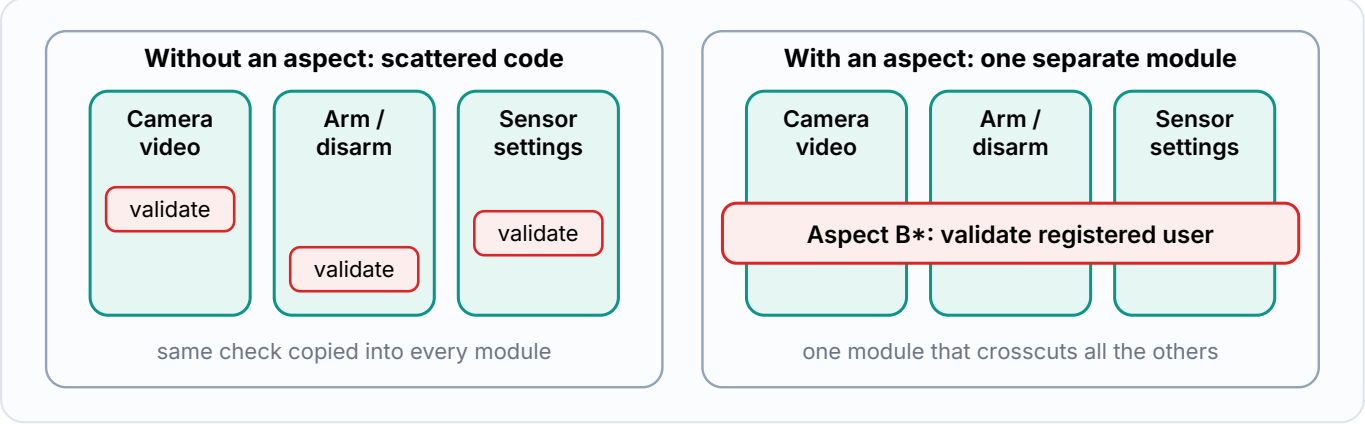


Fig 5.8 A crosscutting concern (user validation) as scattered/tangled fragments (left) versus as a separate aspect module (right). Drawn for this guide from Pressman 8.3.9

10 · Refactoring ★★★ MUST KNOW

> **DEFINITION — LEARN THIS LINE**
> **Refactoring** — "the process of changing a software system in such a way that it **does not alter the external behavior** of the code [design] yet **improves its internal structure**" (Fowler).

An important design activity in many **agile methods** (e.g., Extreme Programming): a reorganization that **simplifies the design (or code) of a component without changing its function or behaviour**. The design is examined for **redundancy, unused design elements, inefficient or unnecessary algorithms, poorly constructed or inappropriate data structures**, or any other failure that can be corrected. Result: software that is **easier to integrate, test and maintain**.

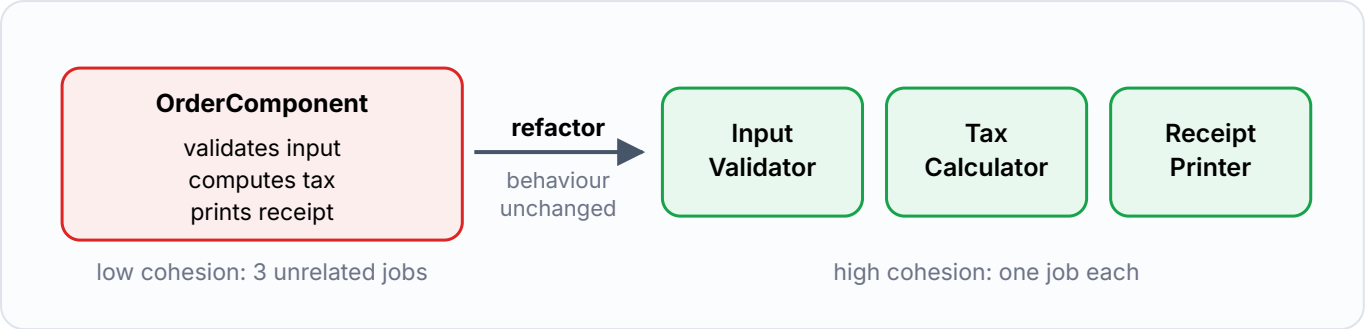


Fig 5.9 The textbook's refactoring case: a component performing three loosely related functions (low cohesion) is refactored into three highly cohesive components. Drawn for this guide from Pressman 8.3.10

Refinement vs refactoring: refinement *adds detail* top-down while the design is being created; refactoring is done *after* a design or code exists — "kind of an optimization pass through the software" — and changes only internal structure. Refactoring does **not** mean redesigning everything iteratively.

11 · Object-oriented design concepts ★★ IMPORTANT

The chapter names **classes and objects, inheritance, messages and polymorphism** and explains them in Appendix 2. Coad and Yourdon: **object oriented = objects + classification + inheritance + communication**.

Class	Encapsulates the **data and procedural abstractions** describing a real-world entity — a template/blueprint for similar objects.
Object	An **instance** of a class; it inherits the class's attributes and operations.
Attributes / operations	Attributes describe the class (values from a domain); operations (**methods, services**) are the algorithms that process the data.
Encapsulation	Data, operations and related information **packaged under one name** and reused as one component.
Inheritance	A **subclass** (specialization) inherits **all attributes and operations** of its **superclass** (generalization) — reuse is direct; superclass changes propagate down.
Messages	A message, *message(<parameters>)*, stimulates behaviour in the receiving object, which executes an operation.
Polymorphism	Lets **different operations have the same name**, reducing the effort to extend a design and decoupling objects.

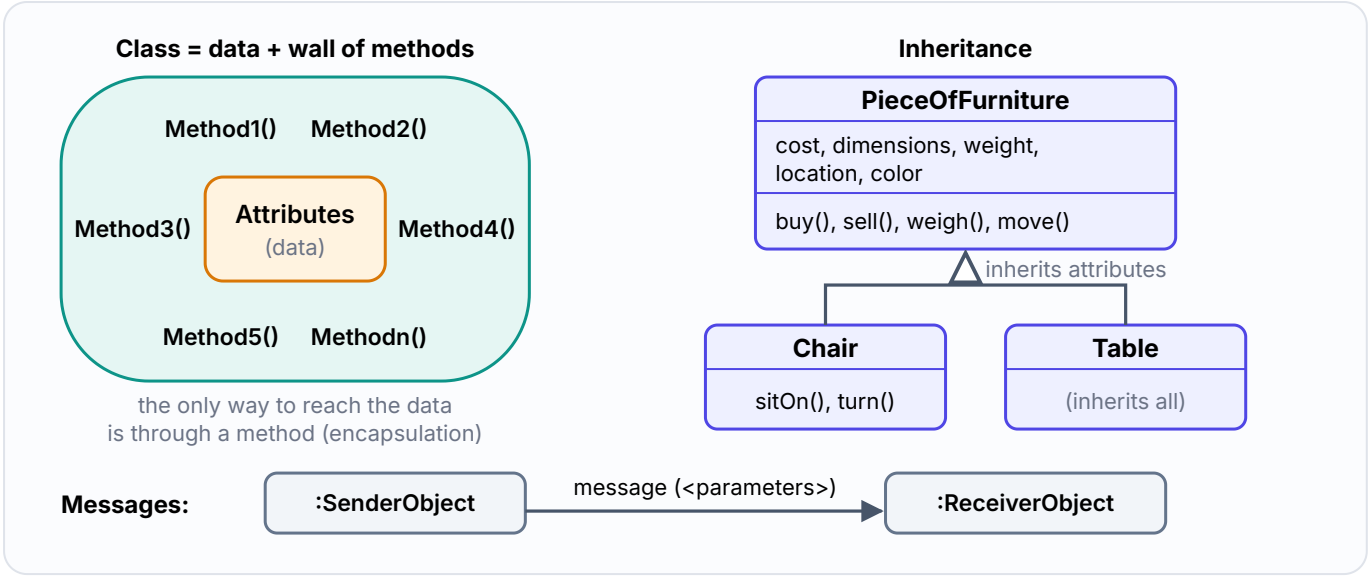


Fig 5.10 Core OO ideas: a class hides its data behind a "wall" of methods; subclasses inherit from a superclass; objects interact by messages. Redrawn from Pressman Appendix 2 (Figs A2.1, A2.2, furniture example)

Polymorphism example: a conventional program that draws line graphs, pie charts, histograms and Kiviat diagrams needs control logic (`if graphtype = linegraph then DrawLineGraph(data)` ...), and every new graph type means a new module *and* edited logic. In OO design each graph is a subclass of **Graph** with its own `draw` operation, so the design reduces to `draw <graphtype>` ; a new type is just a new subclass. A new class can be built from scratch, inherit from a class with most of what it needs, inherit after restructuring the hierarchy, or override existing characteristics; misused inheritance (e.g., multiple inheritance) makes designs error-prone.

> ★ KEY POINT
> The class "wall" **achieves information hiding**; methods working on few attributes **raise cohesion**; communicating only through methods **lowers coupling**. OO concepts reinforce concepts 6 and 7.

12 · Design classes ★★★ MUST KNOW

Analysis classes (from the requirements model) describe problem-domain elements that users see, at a high level of abstraction and in business jargon. **Design classes** (1) **refine the analysis classes** with the detail needed to implement them and (2) implement a **software infrastructure** supporting the business solution — with much more technical detail.

Design class type (Ambler)	Definition — each type is a layer of the design architecture	Example
User interface	All abstractions needed for **human–computer interaction (HCI)**, often visual elements of a metaphor (checkbook, order form, fax machine).	ControlPanelScreen
Business domain	Often **refinements of analysis classes**; the attributes and services (methods) to implement some business-domain element.	Sensor, Camera
Process	Implement **lower-level business abstractions** needed to fully manage the business domain classes.	SensorMonitor
Persistent	Represent **data stores** (e.g., a database) that **persist beyond the execution** of the software.	EventLogStore

Design class type (Ambler)	Definition — each type is a layer of the design architecture	Example
System	Implement **software management and control functions** so the system can operate and communicate within its environment and with the outside world.	NetworkConnection

Well-formed class (Arlow and Neustadt)	Meaning and textbook example
Complete and sufficient	**Complete:** contains all attributes and methods that can reasonably be expected from its name (class *Scene* in video-editing software has everything for creating a scene). **Sufficient:** only the methods needed for its intent — **no more and no less**.
Primitiveness	Each method does **one service**, and the class offers **no second way** to do the same thing (*VideoClip*: only *setStartPoint()* and *setEndPoint()* set start-point and end-point).
High cohesion	A **small, focused set of responsibilities**, single-mindedly implemented (all *VideoClip* editing methods work only on the clip's attributes).
Low coupling	Collaboration kept to an **acceptable minimum**; classes know little of other classes. **Law of Demeter**: a method should only send messages to methods in **neighbouring classes** ("only talk to your friends; don't talk to strangers").

MEMORY TRICK

Five types: "**U**nder **B**usy **P**rofessors, **P**upils **S**tudy" (User interface, Business domain, Process, Persistent, System). Well-formed: "**C**an **P**eople **H**ave **L**unch?" (Complete & sufficient, Primitiveness, High cohesion, Low coupling).

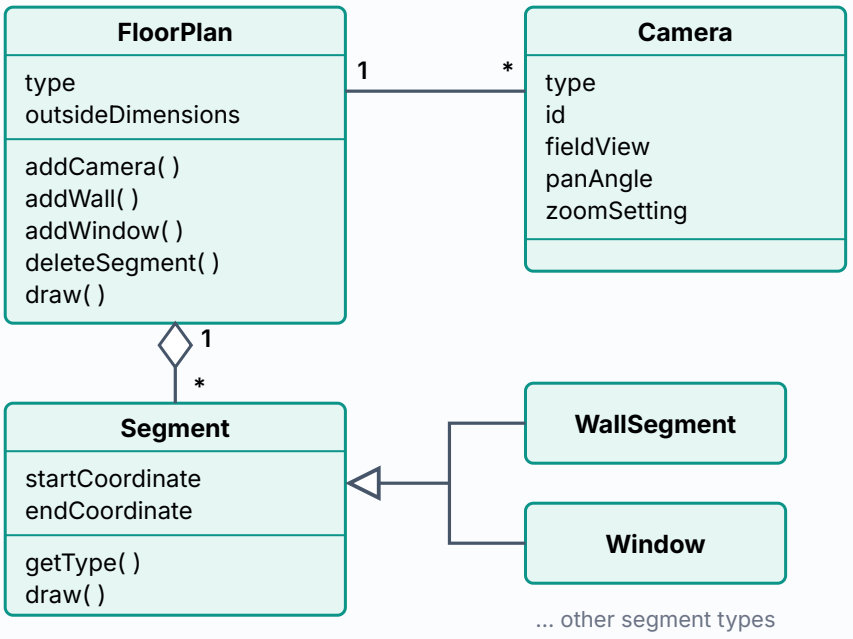


Fig 5.11 The analysis class FloorPlan refined into a design class: FloorPlan is an aggregation of Segments (lists of wall segments, windows, doors ...) and collaborates with many Cameras. New items can be added to or deleted from the aggregation easily. Pressman Fig 8.3

13 · Dependency inversion ★★ IMPORTANT (8th/9th edition)

In hierarchical architectures, high-level "control" components depend **directly** on low-level "worker" components, so changing a worker forces changes in the controller — the design is tightly **coupled**.

> **DEFINITION — LEARN THIS LINE**
>
> **Dependency inversion principle — high-level modules (classes) should not depend [directly] upon low-level modules; both should depend on abstractions. Abstractions should not depend on details; details should depend on abstractions.**

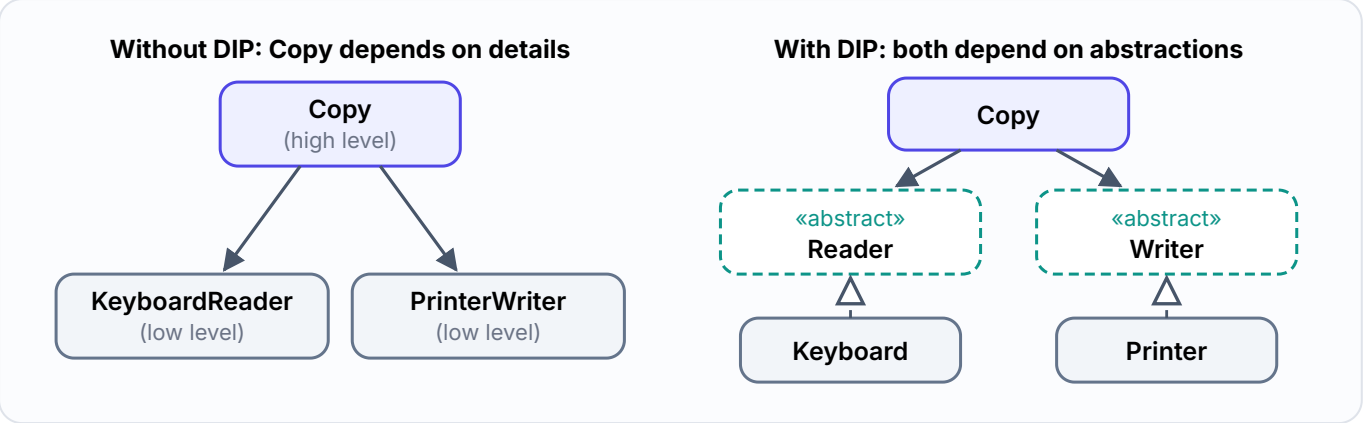


Fig 5.12 Dependency inversion: the control class Copy uses only abstract Reader/Writer classes; concrete Keyboard and Printer classes implement them (dashed arrow, hollow head). Drawn for this guide

The newer edition's illustration is a small program that reads keystrokes and prints them: the control module invokes the workers **through abstractions** (abstract classes). The abstract reader can point to a keyboard's *read()* in one context and a sensor's in another, so workers can be swapped without touching the controller. This **reduces coupling** and **improves testability**.

14 · Design for test ★★ IMPORTANT (8th/9th edition)

There is a chicken-and-egg debate about whether **software design or test-case design should come first**:

- In **test-driven development (TDD)** tests are written **before** other code and guide the design in short cycles: write a test → it fails → write just enough code → the test passes.
- If design comes first, it must include **seams** (also called **test hooks**) — places in the detailed design where test code can be inserted to **probe the state of the running software** and to **isolate the code under test** from its production environment. Seams must be **consciously designed at the component level**, thinking about the tests each component will need — like building a car with diagnostic ports already fitted.

5.4 Design concepts at a glance ★★★ MUST KNOW

Use this as the skeleton for "Explain the fundamental design concepts": one line each, then expand the ones you know best with a diagram.

Concept	One-line meaning	Why it matters
Abstraction	Name what something does, suppress how; procedural and data abstractions.	Think at one level at a time; reusable components.

Concept	One-line meaning	Why it matters
Architecture	Overall structure of the software giving conceptual integrity.	Framework for all detailed design.
Patterns	Named, proven solution to a recurring problem in a context.	Reuse of proven design knowledge; saves time.
Separation of concerns	Split a complex problem into independently solvable pieces.	Less total effort (divide and conquer).
Modularity	Separately named, addressable modules integrated to meet requirements.	Intellectually manageable; easier to plan, test, change.
Information hiding	Each module hides its design decisions (algorithms, data).	Errors made during change do not propagate.
Functional independence	Single-minded modules: high cohesion, low coupling.	Easier to develop, test, maintain, reuse; no ripple effect.
Refinement	Top-down, step-by-step elaboration down to code (Wirth).	Orderly detail; fewer errors and omissions.
Aspects	Representation of a crosscutting concern (e.g., security).	Crosscutting code kept in one module, not scattered.
Refactoring	Better internal structure, same external behaviour.	Simpler; easier to integrate, test, maintain.
OO design concepts	Classes, objects, inheritance, messages, polymorphism.	Encapsulation and reuse; supports hiding and low coupling.
Design classes	Analysis classes refined with technical detail; 5 types; well-formed.	Directly implementable classes and infrastructure.
Dependency inversion	High- and low-level modules both depend on abstractions.	Lower coupling; swappable parts; testability.
Design for test	Design seams/test hooks, or let tests drive design (TDD).	Components can be tested in isolation.

5.5 The design model ★★★ MUST KNOW

The two dimensions of the design model ★★★ MUST KNOW

The **process dimension** shows the **evolution of the design model** as design tasks are executed in the software process. The **abstraction dimension** shows the **level of detail** as each analysis-model element is transformed into a design equivalent and refined iteratively. The **dashed line** is the boundary between the analysis and design models — sometimes clear, sometimes the analysis model slowly blends into the design. Design uses many of the **same UML diagrams** as analysis, but **refined and elaborated** with more implementation-specific detail, emphasizing architectural structure and style, components, and interfaces.

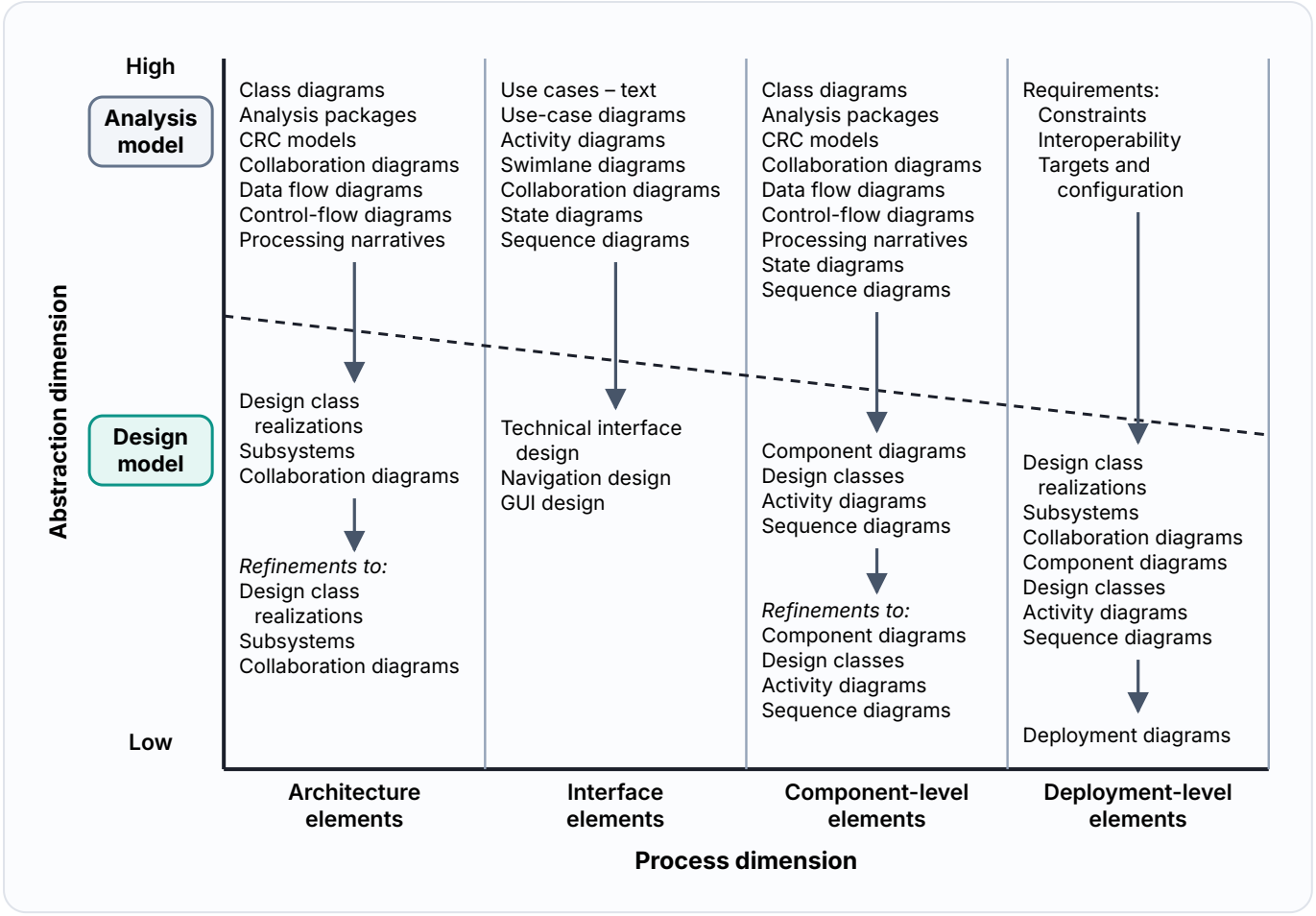


Fig 5.13 Dimensions of the design model. Right = the design evolves through the process; down = more detail. The dashed line separates the analysis model (above) from the design model (below). Pressman Fig 8.4

The elements on the horizontal axis are **not always developed in sequence**: preliminary **architectural design** usually comes first, then **interface** and **component-level design**, often **in parallel**; the **deployment model** is usually delayed until the design is fully developed. **Design patterns** can be applied at any point.

> ✎ **EXAM TIP**
>
> To draw it in the exam: two axes (abstraction high→low, process →), four columns, a sloping dashed line with "analysis model" above and "design model" below, and 2–3 diagram names per column with downward arrows.

Data design elements ★★ IMPORTANT

Data design (also called **data architecting**) builds a model of data/information at a **high level of abstraction** (the customer's/user's view), then refines it into progressively more **implementation-specific** representations. The **architecture of the data strongly influences the architecture of the software** that processes it. Data design matters at every level:

- **Program component level** — design of **data structures** and the **algorithms** that manipulate them.
- **Application level** — translation of the **data model** into a **database**, pivotal to the business objectives.
- **Business level** — data from disparate databases reorganized into a **data warehouse** for **data mining / knowledge discovery**.

Architectural design elements ★★★ MUST KNOW

Equivalent to the **floor plan of a house** — the layout of rooms, their size, shape and relationships, and the doors and windows that allow movement — architectural elements give an **overall view of the software**. The architectural model is derived from **three sources**:

1. information about the **application domain** of the software;
2. specific **requirements model elements** (e.g., data flow diagrams or analysis classes, their relationships and collaborations);
3. the availability of **architectural styles and patterns**.

It is usually drawn as a **set of interconnected subsystems**, often derived from analysis packages; each subsystem may have its own architecture (e.g., a GUI following a standard UI style).

Interface design elements ★★★ MUST KNOW

Like the **detailed drawings for the doors, windows and external utilities** of a house (how they operate, how water, power, gas and telephone come in and are distributed, where the doorbell and intercom are), interface elements show **how information flows into and out of the system** and **among the components** of the architecture. There are three:

Interface element	Key points
1. User interface	Now often called **usability design**. Includes **aesthetic** elements (layout, colour, graphics, interaction mechanisms), **ergonomic** elements (information layout and placement, metaphors, UI navigation) and **technical** elements (UI patterns, reusable components). The UI is usually a **unique subsystem** of the architecture.
2. External interfaces	To other **systems, devices, networks** or producers/consumers of information. Need definitive information about the other entity — gathered in requirements engineering and **verified** when interface design starts (it can change over time). Should include **error checking** and, if needed, **security features**.
3. Internal interfaces	Between **design components**; closely tied to component-level design. Design classes define the operations and **messaging schemes** for collaboration; each message carries the needed information for the requested operation. In input–process–output design, interfaces come from data flow and processing narratives.

DEFINITION — LEARN THIS LINE

Interface (UML) — "a specifier for the externally-visible [public] operations of a class, component, or other classifier (including subsystems) without specification of internal structure" — simply, a **set of operations** describing part of a class's behaviour and giving access to them.

In an advanced SafeHome, control-panel functions may be offered through a wireless PDA or mobile phone. **ControlPanel** implements *readKeyStroke()* and *decodeKey()*; to provide them to **WirelessPDA** and **MobilePhone**, an interface **KeyPad** is defined with **no attributes**, only the keypad operations. In UML this is a **realization**: part of ControlPanel's behaviour is implemented by realizing KeyPad's operations.

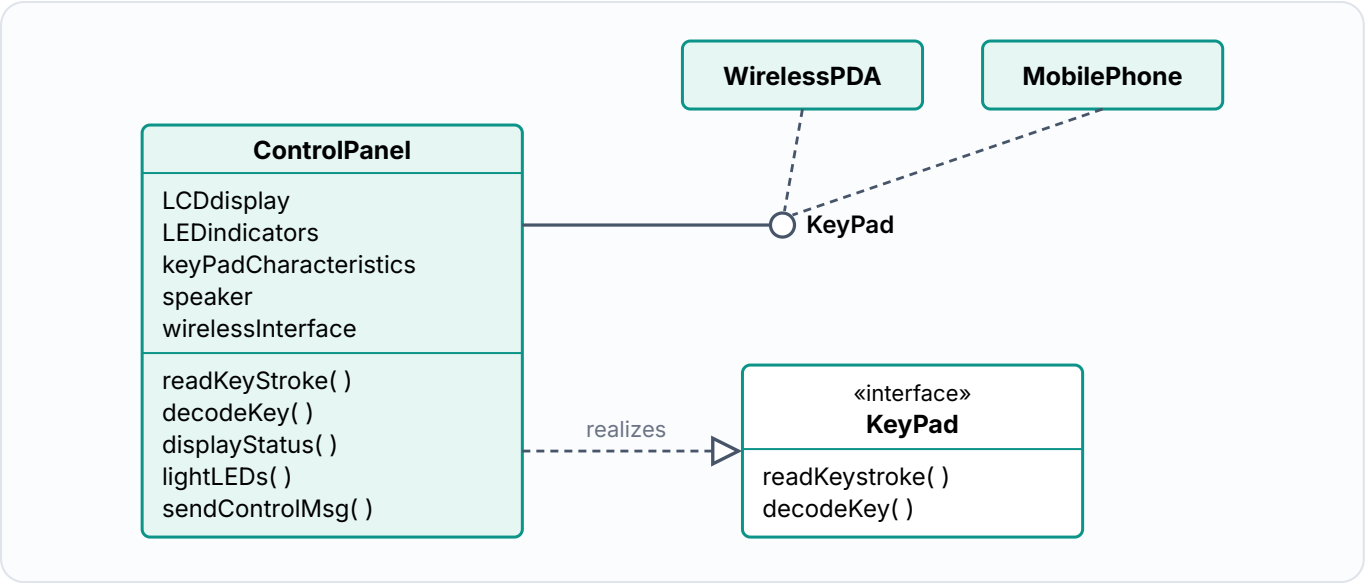


Fig 5.14 Interface representation for ControlPanel: KeyPad shown as a small labelled circle and as an «interface» box. The dashed line with an open triangle means ControlPanel *realizes* KeyPad, whose operations WirelessPDA and MobilePhone use. Pressman Fig 8.5

Component-level design elements ★★★ MUST KNOW

Like the **detailed drawings of each room** (wiring, plumbing, switches, sinks, cabinets, flooring), component-level design **fully describes the internal detail of each component**: **data structures** for all local data objects, **algorithmic detail** for all processing, and an **interface** giving access to all component operations. In OO work a component is drawn as a **UML component diagram** (Fig 5.15a): **SensorManagement** performs all SafeHome sensor functions (monitoring, configuring) and a dashed arrow links it to its assigned class **Sensor**. Details can be modelled with a **UML activity diagram** (processing logic), **pseudocode** or a **flowchart/box diagram** (procedural flow, following **structured programming** rules), and data structures in pseudocode or the implementation language.

Deployment-level design elements ★★ IMPORTANT

They show **how software functionality and subsystems are allocated within the physical computing environment**. SafeHome runs on a **home-based PC**, the **SafeHome control panel** and a **server at CPI Corp.** (Internet access). A **UML deployment diagram** (Fig 5.15b) shows the subsystems in each node — the PC houses security, surveillance, home management and communication, plus an **external access** subsystem for all outside access attempts. It begins in **descriptor form** (environment in general terms — "personal computer" could be a Mac, Windows PC, Sun workstation or Linux box) and is later redrawn in **instance form** (each specific, named hardware configuration identified) late in design or as construction begins.

All design model elements in one table ★★★ MUST KNOW

Element	House analogy	What it describes	Typical notation
Data	(the foundation)	Data model refined from the user's view into data structures, databases, data warehouses	Class diagrams, data structures
Architectural	Floor plan	Overall structure: interconnected subsystems, styles, patterns	Subsystems, design class realizations
Interface	Doors, windows, external utilities	Information flow in/out and between components: UI, external, internal	GUI/navigation design, «interface»

Element	House analogy	What it describes	Typical notation
Component-level	Detailed drawings of each room	Local data structures, algorithms, interface of each component	Component, activity diagrams, pseudocode
Deployment-level	(where each part is placed)	Allocation of subsystems to physical computing nodes	Deployment diagrams

⚠ **WATCH OUT**

The textbook counts the elements differently in different places: a margin note gives four (**data, architecture, components, interface**); the summary gives four (**architectural, interface, component-level, deployment-level**); the headings give **five**. If asked for "the elements of the design model", write all five.

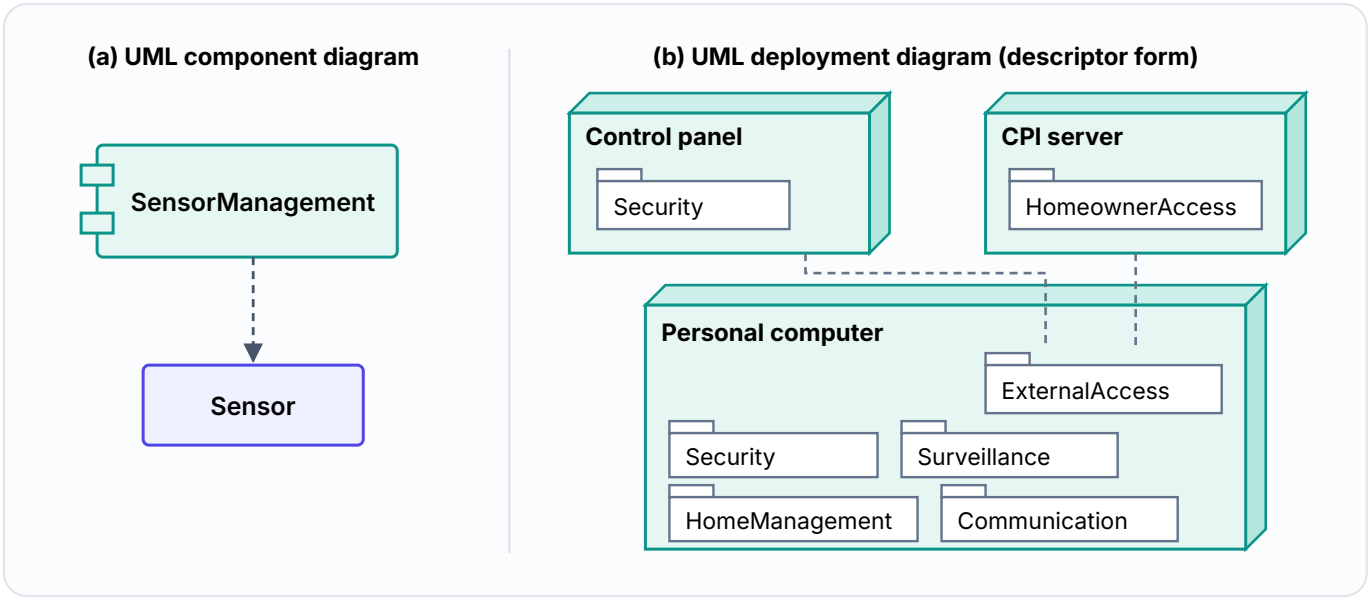


Fig 5.15 (a) UML component diagram: SensorManagement with its assigned class Sensor. (b) UML deployment diagram in descriptor form: three computing environments and the subsystems each houses. Pressman Figs 8.6 and 8.7

Chapter 5 in one page

Core ideas

- **Design model** = architecture, data structures, interfaces, components (requirements model = data, function, behaviour).
- **Why design?** Quality is fostered here: assessable, the only accurate translation of requirements, foundation for later work; without it the system is unstable.
- **Position**: last action of modeling, before construction, in any process model. Begin with data, then architecture.
- **Good design**: firmness, commodity, delight; diversification → convergence; iterative, high → low abstraction.

Lists to write word-perfect

- **3 characteristics**: explicit + implicit requirements; readable guide; complete picture (data, function, behaviour).
- **FURPS**: Functionality, Usability, Reliability, Performance, Supportability.
- **Methods share**: translation mechanism, notation, refinement/partitioning heuristics, quality guidelines.
- **Architecture**: structural, extra-functional, families of related systems; models: structural, framework, dynamic, process, functional.
- **Design classes**: user interface, business domain, process, persistent, system.
- **Well-formed**: complete & sufficient, primitiveness, high cohesion, low coupling (Law of Demeter).
- **Architecture sources**: domain, requirements elements, styles & patterns. **Interfaces**: UI, external, internal.

The concepts in one line each

- **Abstraction**: hide detail; procedural (*open*), data (*door*).
- **Architecture**: overall structure, conceptual integrity.
- **Patterns**: proven solution, recurring problem, context.
- **Separation of concerns**: divide and conquer.
- **Modularity**: named modules; stay near *M*.
- **Information hiding**: modules hide decisions (Parnas).
- **Functional independence**: high cohesion, low coupling.
- **Refinement**: top-down elaboration (Wirth).
- **Aspects**: crosscutting concern in its own module.
- **Refactoring**: better structure, same behaviour (Fowler).
- **OO**: class, object, inheritance, message, polymorphism.
- **Design classes**: 5 types, 4 well-formed traits.
- *Newer*: **dependency inversion**, **design for test**.

Diagrams to practise

- Fig 5.2 requirements → design pyramid
- Fig 5.4 modularity–cost graph
- Fig 5.6 cohesion / coupling
- Fig 5.13 design-model dimensions

Abstraction vs refinement

Abstraction hides low-level detail from outsiders; refinement reveals it step by step. Complementary.

Cohesion vs coupling

Cohesion = strength **inside** a module (high). Coupling = dependence **between** modules (low; no ripple effect).

Likely exam questions

2M **Define software design. Why is design called "the place where quality is fostered"?**
How to answer → Design model definition; representations can be assessed for quality before coding (5.1).

2M **What is procedural abstraction and data abstraction? Give an example of each.**
How to answer → Textbook definitions + *open* / *door* (Fig 5.3).

2M **Expand FURPS and state how each attribute is assessed.**
How to answer → FURPS table (5.2); supportability includes maintainability.

5M **Explain how the requirements model is translated into the design model, with a neat diagram.**
How to answer → Fig 5.2 → one line per design model and its input → "begin with data".

5M **What are the characteristics of a good design? List the design quality guidelines.**
How to answer → McGlaughlin's three characteristics + the eight guidelines (5.2).

10M **Explain the fundamental software design concepts.**
How to answer → Three questions they answer → 5.4 table as skeleton → expand abstraction, modularity (graph), information hiding, functional independence, refinement with small diagrams.

5M **Differentiate between abstraction and refinement.**
How to answer → Definitions (Booch / Wirth) → comparison table in concept 8 → "complementary".

6M **What is functional independence? Explain cohesion and coupling, and why high cohesion and low coupling are desired.**
How to answer → Definitions → Fig 5.6 → comparison table → benefits (develop, test, maintain, reuse, no ripple effect). If "types" are asked, add the coupling categories / cohesion scale.

6M **Explain modularity. Show the relation between number of modules and software cost with a graph.**
How to answer → Definition + Myers → Fig 5.4 (three curves, *M*) → avoid under-/over-modularity → five reasons.

4M **Explain information hiding and its benefits.**
How to answer → Parnas definition → Fig 5.5 → errors during modification don't propagate.

4M **What are aspects and crosscutting concerns? Explain with an example.**
How to answer → Concern → crosscut → aspect → SafeHomeAssured.com A*/B* → Fig 5.8.

4M **What is refactoring? How does it differ from refinement?**
How to answer → Fowler definition → what is examined → Fig 5.9 → refinement vs refactoring.

6M **Explain the five types of design classes and the characteristics of a well-formed design class.**
How to answer → Analysis vs design class → 5 types → 4 traits with Scene/VideoClip and the Law of Demeter.

10M **Explain the dimensions of the design model with a diagram, and describe each design model element.**
How to answer → Fig 5.13 → data, architectural (3 sources), interface (3 parts, Fig 5.14), component-level, deployment-level (Fig 5.15) with house analogies.

EXAM DAY

Exam-Day Playbook

How to turn what you studied into marks: answer structure, command words, diagrams and the comparison questions to expect.

Writing answers that score

2 marks

- One-line **definition** in textbook words
- Plus **one** example or key feature
- 2–3 lines total. Don't write an essay.

4–6 marks

- Definition
- A small diagram or a table, if one exists
- 4–6 points, each with a bold keyword and one explaining line
- One example

8–10 marks

- Definition / introduction (2 lines)
- **Neat labelled diagram**, drawn first
- Explain each box or phase under its own sub-heading
- Advantages and disadvantages, or a comparison
- When to use it + a case-study example
- One-line conclusion

★ KEY POINT

Examiners mark **keywords**, **structure** and **diagrams**. Underline the keywords (the highlighted phrases in this guide), number your points, and leave space between them.

Diagram rules

1. **Draw first, write after.** The diagram tells you what to explain, and it scores even if time runs out.
2. **Label every box and arrow** using the textbook's names (for example "Requirements definition", not "Req").
3. **Give it a caption**: *Fig: The waterfall model.*
4. **Use boxes for activities, arrows for flow**, and dashed arrows for feedback or iteration.
5. **Keep it big and clean.** Half a page is fine for a 10-mark question.

Command words decoded

If the question says…	…it wants
Define / What is	The exact definition, plus one example. Short.
List / Enumerate	The complete list, numbered. One line each is enough.

If the question says…	…it wants
Explain / Describe	Definition + points with explanation + diagram if one exists + example.
With a neat diagram	The diagram carries a large share of the marks. Draw it carefully and explain every part.
Differentiate / Compare / Distinguish	A **table** with a "Basis" column and 5–7 rows. Add one line each of "use A when…, use B when…".
Discuss / Justify	Points for and against, then your conclusion with a reason.
Apply to / For the given case study	Name the system in every point. Generic answers lose marks here.

✎ **CASE-STUDY QUESTIONS**

When a question names a system (insulin pump, Mentcare, weather station, iLearn, or something new like a library or ATM system), connect every point to it: "*Because the insulin pump is safety-critical, …*". If the system is new, pick the textbook system it most resembles and reuse that reasoning.

Diagrams you must be able to draw

Every textbook or slide diagram in this guide. Tick each one when you can draw it from memory with all labels. Diagrams drawn only to help understanding are not listed.

Ch 1 · Introduction

☐ **Fig 1.3** The four essential attributes of good software

☐ **Fig 1.4** Software engineering is one part of system engineering and builds on computer science

☐ **Fig 1.5** Insulin pump hardware architecture

☐ **Fig 1.6** Activity model of the insulin pump (UML)

☐ **Fig 1.7** Organisation of the Mentcare system

☐ **Fig 1.8** The weather station's environment (UML packages with the «system» stereotype)

☐ **Fig 1.9** The iLearn architecture

Ch 2 · Software Processes

☐ **Fig 2.1** The waterfall model

☐ **Fig 2.2** Incremental development

☐ **Fig 2.3** Reuse-oriented software engineering

☐ **Fig 2.4** The V-model as drawn by Pressman

☐ **Fig 2.5** The prototyping paradigm

☐ **Fig 2.6** A typical spiral model

☐ **Fig 2.7** The requirements engineering process

☐ **Fig 2.8** A general model of the design process

☐ **Fig 2.9** Stages of testing (slide 28 labels the last stage "Acceptance testing")

☐ **Fig 2.10** Testing phases in a plan-driven software process (V-model)

☐ **Fig 2.11** System evolution

☐ **Fig 2.12** The process of prototype development

Ch 4 · Requirements Engineering

☐ **Fig 4.1** User and system requirements

☐ **Fig 4.2** Readers of different types of requirements specification

☐ **Fig 4.3** The eight stakeholder groups of the Mentcare system (the textbook also includes…

☐ **Fig 4.4** One non-functional requirement can generate new functional requirements and restrict…

☐ **Fig 4.5** Types of non-functional requirement

☐ **Fig 4.7** A spiral view of the requirements engineering process

☐ **Fig 4.8** The requirements elicitation and analysis process (step 4 is called "requirements…

☐ **Fig 4.9** Ethnography and prototyping for requirements analysis

☐ **Fig 4.10** Use cases for the Mentcare system

☐ **Fig 4.11** Users of a requirements document and how they use it

☐ **Fig 4.12** Requirements evolution

☐ **Fig 4.13** Requirements change management

☐ **Fig 4.14** What the insulin pump software does, with the textbook requirements that govern each step

Ch 5 · Design Concepts

☐ **Fig 5.1** Design is the last action of modeling

☐ **Fig 5.2** Translating the requirements model into the design model

☐ **Fig 2.13** Incremental delivery

☐ **Fig 2.14** The process improvement cycle

☐ **Fig 2.15** Capability maturity levels as a staircase (box names from slide 54; grey meanings from...

Ch 3 · Agile

☐ **Fig 3.1** Plan-driven and agile development

☐ **Fig 3.3** The XP release cycle

☐ **Fig 3.4** A "prescribing medication" story card

☐ **Fig 3.5** Examples of task cards for prescribing medication (Task 3 shown in full)

☐ **Fig 3.8** Test case description for dose checking

☐ **Fig 3.9** The Scrum sprint cycle

☐ **Fig 3.11** Distributed Scrum

☐ **Fig 3.12** Factors influencing the choice of plan-based or agile development

☐ **Fig 5.3** The textbook's door example

☐ **Fig 5.4** Modularity and software cost

☐ **Fig 5.8** A crosscutting concern (user validation) as scattered/tangled fragments (left) versus...

☐ **Fig 5.9** The textbook's refactoring case

☐ **Fig 5.10** Core OO ideas

☐ **Fig 5.11** The analysis class FloorPlan refined into a design class

☐ **Fig 5.13** Dimensions of the design model

☐ **Fig 5.14** Interface representation for ControlPanel

☐ **Fig 5.15** (a) UML component diagram

Ch 6 · Architectural Design

☐ **Fig 6.2** The architecture of a packing robot control system

☐ **Fig 6.4** Architectural design decisions

☐ **Fig 6.5** The five architectural strategies at a glance, and the textbook's example of a conflict

☐ **Fig 6.6** Krutchen's 4+1 view model